AOSP 进阶丛书 · 第三册 · ANDROID 13 (API 33)

Binder IPC 全链路剖析

跟随 App 调用 AMS 的一个方法穿过十跳：mmap 一次拷贝原理、BC/BR 命令协议、servicemanager 名录与门禁、AIDL 生成物解剖、线程池与死亡通知——跨进程调用从此白盒。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：一个调用打穿整个系统 2
第一章 为什么是 Binder 2
第二章 总体架构：五层纵切 4
第三章 驱动协议：BC/BR 与核心结构 5
第四章 ProcessState：进程的 Binder 资产 8
第五章 IPCThreadState：命令收发中枢 8
第六章 Parcel：跨进程的类型系统 11
第七章 servicemanager：名录与门卫 12
第八章 Java 层与 AIDL：对象的两个世界 13
第九章 端到端全链路：把图走成路 19
第十章 线程池与高级机制 20
第十一章 设计决策十二问 23
第十二章 深水区：Binder 观测武器库与 RPC 后端 23
附录 A 观测实战 24
附录 B 三册联动：一条完整的因果链 25
附录 C 自检题 25
附录 D 术语速查 25
附录 E 动手实验（一天量） 26
第十三章 Parcel 线格式：字节级走读 26
第十四章 错误路径：DEAD、FAILED 与状态码通道 30
第十五章 三域、binderfs 与稳定性等级 34
第十六章 死亡通知全链路与重连模式库 38
教科书习题集（分级） 43
术语表（按拼音排序） 46
排障手册：Binder 问题定位 47

本册速览

一次拷贝的 mmap 原理与 1MB-2 页账本 · BC/BR 命令协议
端到端十跳（App→驱动→AMS） · 服务注册五道关卡 · AIDL 生成物解剖
线程池 15+1/31 · 死亡通知 · oneway 真实语义 · 冻结 BR_FROZEN_REPLY
深水区：三层观测武器库与 RPC 后端
先读第一章对比表再入十跳；附录 E 的 MiniBinder 动手做一遍

前言：一个调用打穿整个系统

第一册说“一切跨层皆 Binder”，第二册让 system_server 从 zygote 里站了起来——本册补上最后一块：**进程们如何对话**。你将跟随 **App 调用 AMS 的一个方法**，从 Java 门面走进 JNI、穿过 libbinder、越过内核驱动、落到服务端线程栈，再原路返回。读完本册，“跨进程调用”对你将不再是黑盒，而是十跳可以逐一命名的白盒。

源码基线同前两册（AOSP Android 13）。特别说明：本树 `kernel/` 不含驱动源码（binder.c 在独立内核仓），驱动侧内容依据稳定 UAPI 头 `external/kernel-headers/original/uapi/linux/android/binder.h` 与 libbinder 的调用序列反推，图会明确标注这一点。

第一章 为什么是 Binder

> **设计洞察 · 学习目标**
> 1. 对比四种IPC的账单；2. 解释一次拷贝的原理；3. 画出mmap映射图

1.1 四种 IPC 的对账单

机制	拷贝次数	安全模型	同步语义	能力传递
管道/Unix socket	2 次	UID 自报（可伪造为 root 视角）	阻塞读写	无
SysV/POSIX shm	0 次	密钥约定，无内核仲裁	无（自建）	无
信号量/消息队列	1-2 次	IPC 权限位	弱	无
Binder	**1 次**	**内核采集 euid/pid + SELinux 上下文**	同步/oneway/死亡通知/引用传递	**对象能力（binder node/ref）**

Binder 从不是“最快的”（共享内存零拷贝），但它是**每微秒买到最多语义**的：一次拷贝顺带换来了能力模型（跨进程的对象引用）、内核背书的身份、死亡通知与有序投递。Android 的一切权限检查都建立在这张账单上。

1.2 一次拷贝是怎么做到的

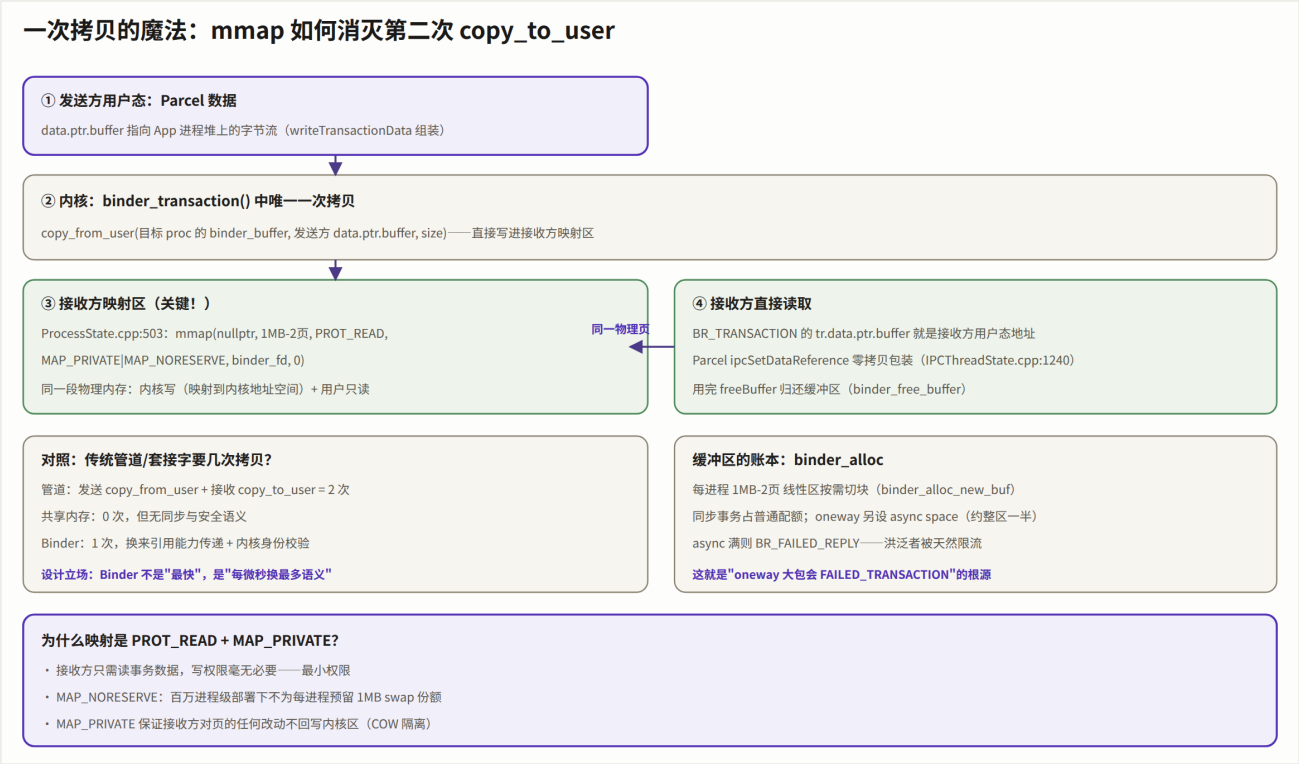


图 1 mmap 一次拷贝示意图：接收方 1MB-2 页只读映射区、copy_from_user 直写、缓冲账本与三条 mmap 参数的设计动机

关键在于 **mmap 的方向**：接收进程对 `/dev/binder` 调 mmap（`ProcessState.cpp:503`）后，内核把该进程的接收缓冲区同时映射进内核地址空间与该进程用户态（只读）。发送时驱动 `copy_from_user` 一次就把数据写进这块双重映射区——**接收方读自己的映射就拿到了数据，无需第二次拷出**。对照图右下：`PROT_READ`（接收方不需要写）、`MAP_NORESERVE`（百万进程不为每份预留 swap）、`MAP_PRIVATE`（写时复制隔离）——三个参数各自是一道安全或内存题的答案。

1.3 血统：从 OpenBinder 到 binderfs

Binder 不是 Linux 亲生子：它的直系祖先是 Palm 的 OpenBinder（Dianne Hackborn 主导），2005 年随 Android 落户 Linux 内核（`drivers/android/binder.c`），此后主线化抗争十几年——至今仍以“Android 专属驱动”身份活在 staging 之外。2019 年的 binderfs（本树 UAPI `binderfs.h`）让容器/多用户环境能按需实例化 binder 设备节点，三域（binder/hwbinder/vndbinder）从此可以动态分配而非写死三个 misc 设备。读系统代码时懂一点血统史，能解释很多“为什么它长这样”的怪癖——Binder 的面向对象句柄模型、上下文管理者设计，都是 OpenBinder 时代就定下的基因。

1.4 安全模型对比：为什么“能传对象”如此重要

传统 IPC 的权限模型止步于“连上了没有”；Binder 的能力模型（capability）把**对象引用本身当作权限凭证**：拿到某服务的 IBinder，才有资格调用它；而 UID/PID 校验、SELinux 上下文随**每一次事务**重新采集。两相结合，Android 的安全叙事完整成立——“你能不能调”由名录与 SELinux 决定，“你是谁”由驱动决定，“你能干什么”由服务端代码决定。理解这三层职责切分，是审阅任何系统服务安全设计的起点。

源码锚点 · 第一章锚点

ProcessState.cpp:47 BINDER_VM_SIZE = 1MB - 2×PAGE_SIZE
ProcessState.cpp:48 DEFAULT_MAX_BINDER_THREADS = 15
ProcessState.cpp:503 mmap(nullptr, BINDER_VM_SIZE, PROT_READ, MAP_PRIVATE|MAP_NORESERVE, fd, 0)
external/kernel-headers/original/uapi/linux/android/binder.h UAPI 协议定义

第二章 总体架构：五层纵切

设计洞察 · 学习目标

1. 说出三个'每X一个'；2. 画出五层纵切图；3. 理解servicemanager角色



图 2 Binder 分层架构：客户端 Java/Native、内核驱动、服务端 Native/Java 的五层纵切与 ProcessState/IPCThreadState/servicemanager 三大件

三个“每 X 一个”记住主体结构：**每进程一个 ProcessState**（开驱动、mmap、线程池策略）、**每线程一个 IPCThreadState**（mIn/mOut 缓冲与命令收发）、**每服务端点一个 binder_node**（对端表现为 binder_ref 句柄）。servicemanager 是唯一的“名录”，句柄 0 固定指向它。

2.1 同步与 oneway 的一图两读

同一张五层图读两遍：同步调用的回程走⑨⑩（reply 原路返回）；oneway 在⑤把 `TF_ONE_WAY` 交给驱动后，⑥的入队走目标的 async_todo（对同目标保序、受 async 配额约束），发送方收到 `BR_TRANSACTION_COMPLETE` 即返回——**两条路的分歧点只有 flags 一个比特**。协议层的克制（不引入第二种事务结构）换来了实现层的统一：所有事务同构，仅靠队列与配额区分行为。

第三章 驱动协议：BC/BR 与核心结构

> **设计洞察 · 学习目标**
> 1. 列举BC/BR命令对；2. 解释五种数据结构关系；3. 描述transaction_stack压栈

3.1 命令对

Binder 协议是“双半工命令流”：应用写给驱动的 `BC_*` 与驱动推给应用的 `BR_*` 在同一个 `ioctl(BINDER_WRITE_READ)` 里双向捎带（`binder_write_read` 结构含 write_buffer/read_buffer 两半）。最重要的命令对：

BC（应用→驱动）	作用	对应 BR（驱动→应用）
BC_TRANSACTION	发起事务（同步或 oneway）	BR_TRANSACTION / BR_REPLY
BC_REPLY	服务端回包	BR_REPLY（唤醒原线程）
BC_INCREFS / BC_ACQUIRE / BC_RELEASE / BC_DECREFS	引用计数维护	BR_INCREFS / BR_ACQUIRE / BR_RELEASE / BR_DECREFS
BC_REQUEST_DEATH_NOTIFICATION	登记死亡通知	BR_DEAD_BINDER / BR_CLEAR_DEATH_NOTIFICATION_DONE
BC_ENTER_LOOPER / BC_REGISTER_LOOPER / BC_EXIT_LOOPER	线程池声明	BR_SPAWN_LOOPER（驱动要求扩池）
—（驱动主动）	—	BR_DEAD_REPLY / BR_FAILED_REPLY / BR_FROZEN_REPLY / BR_TRANSACTION_COMPLETE

特殊返回码就是排障词典：`BR_DEAD_REPLY`（对端进程死亡）、`BR_FAILED_REPLY`（事务被拒——常见于 async 空间耗尽）、`BR_FROZEN_REPLY`（目标进程被缓存冻结，`waitForResponse` :931 快速失败）。

3.2 数据结构关系

3.3 同步事务的驱动侧十步（文字展开）

结合 UAPI 与 libbinder 行为，`binder_transaction()` 处理一笔 BC_TRANSACTION 的路径可以展开为：

- **1 解析**：从 write_buffer 读出 `binder_transaction_data`（target.handle / code / flags / data.ptr / offsets.ptr）。
- **2 寻址**：handle=0 直取上下文管理者（servicemanager）；否则在当前 proc 的 refs 里查 desc → 目标 `binder_node`。
- **3 权限与上下文**：记录 `sender_pid / sender_euid`（凭 struct cred），附加 SELinux security_ctx 供 BR_TRANSACTION_SEC_CTX 使用。
- **4 分配**：在**目标进程**的 binder_alloc 里切一块 `binder_buffer`（同步走普通配额，oneway 先查 async_space）。
- **5 拷贝**：**copy_from_user(目标缓冲, data.ptr.buffer, size)**——全链路唯一一次拷贝；offsets 表里的每个 `flat_binder_object` 顺带翻译（本地 binder → 对端 handle，并为目标建 ref）。
- **6 线程选择**：目标 proc 找空闲 binder 线程；全忙且未超预算则唤醒待命线程并后续发 `BR_SPAWN_LOOPER`；无可用则入 proc todo 队列。
- **7 压栈**：同步事务把 `binder_transaction` 压入目标线程的 `transaction_stack`（reply 将沿栈返回原线程）。
- **8 唤醒**：目标线程 wait 队列唤醒；发送线程继续在 ioctl 里等读（写完命令未完事）。
- **9 回执**：发送线程读到 `BR_TRANSACTION_COMPLETE`；同步事务继续等 `BR_REPLY`，oneway 到此完结。
- **10 服务端收件**：目标线程读出 `BR_TRANSACTION(_SEC_CTX)` 与数据指针（就在它自己的 mmap 区），进入第五章 5.2 的分发。

把这十步与时序图（第九章）互相印证，驱动黑盒就变成了 checklist。

3.4 驱动核心数据结构关系图解（教科书画级）

text 五种数据结构的挂接关系（以 App→AMS 为例）

```
App 进程（客户端）  system_server 进程（服务端）
┌──────────────────┐  ┌──────────────────────┐
| binder_proc |  | binder_proc |
| ├─threads(RB) |  | ├─threads(RB) |
| | └─binder_thread|  | | └─binder_thread |
| | └─transaction_stack |  | └─transaction_stack|
| | └─binder_transaction|  | └─binder_transaction|
| ├─refs_by_desc |  | ├─nodes(RB) |
| | └─binder_ref |  | └─binder_node |
| | desc=handle |  | ptr/cookie →──┼──→ BBinder(JavaBBinder)
| | node ─────────┼──跨进程──→| refs ←─────────┼── binder_ref 挂这
```

```
| └─buffers | | └─buffers |
| (mmap 1MB-2页)| | (mmap 1MB-2页) |
└──────────────┘ └──────────────┘

// 讲解注释①：App 侧拿到的是 binder_ref（desc=32位句柄）
// Java 层包装为 BpBinder → BinderProxy
// 讲解注释②：AMS 侧注册的是 binder_node（ptr/cookie指向 JavaBBinder）
// Java 层就是 Binder.onTransact 的接收者
// 讲解注释③：驱动负责两端的 ref↔node 映射——
// App 的 handle 在驱动里找到 AMS 的 node → 路由到正确进程
```

设计洞察 · 逐段精读：为什么这五种结构是"最小完备集"

proc（进程边界）、thread（执行流+调用栈）、node（服务端点）、ref（客户端引用）、buffer（数据载体）——**每种对应一个跨进程通信的必要维度**。少任何一种，通信就不完整：没有 thread 就无法并发、没有 ref 就无法寻址、没有 buffer 就没有数据。这是"数据结构=问题域的最小模型"的教科书案例。

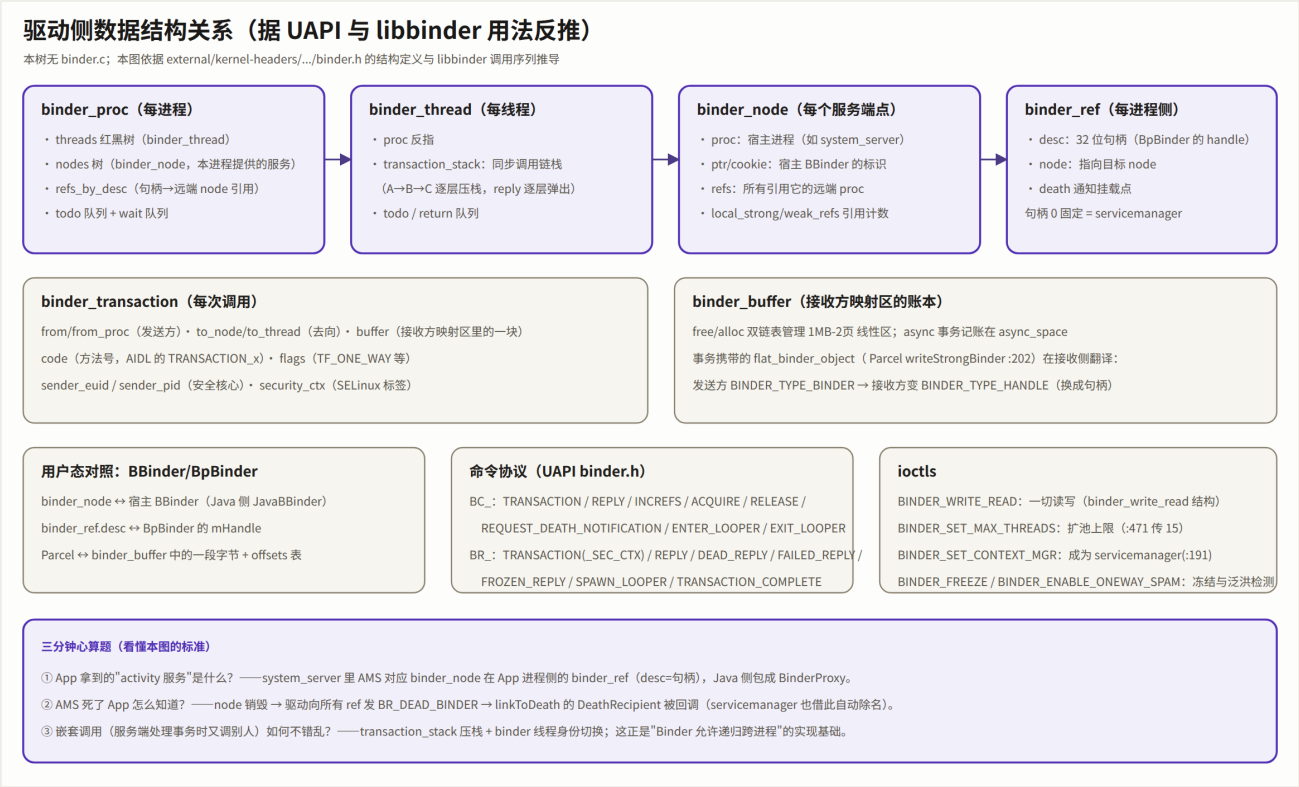


图 3 驱动侧数据结构关系：proc/thread/node/ref/buffer/transaction 的挂接方式与用户态 BBinder/BpBinder 对照（UAPI 反推）

三个最值得内化的关系：**句柄的本质**是接收进程侧 `binder_ref.desc`，Java 世界的 `BinderProxy` 不过是它的马甲；**同步调用链**靠每个线程的 `transaction_stack` 压栈维持，因此 binder 允许跨进程递归（A→B→C 再逐层返回）；**死亡通知**挂在 ref 上，node 销毁时驱动向所有引用方广播 `BR_DEAD_BINDER`——第七章你会看到 servicemanager 如何用这一机制自动除名。

第四章 ProcessState：进程的 Binder 资产

> **设计洞察 · 学习目标**
> 1. 记住BINDER_VM_SIZE与默认线程数两个常量；2. 理解open_driver的协议握手；3. 解释扩池决策权为何在驱动；4. 说出句柄0的起点作用

cpp frameworks/native/libs/binder/ProcessState.cpp:485-517（节选）

```
ProcessState::ProcessState(const char* driver)
: mDriverFD(-1), mVMStart(MAP_FAILED),
mMaxThreads(DEFAULT_MAX_BINDER_THREADS), ... {
base::Result<int> opened = open_driver(driver); // open("/dev/binder")
if (opened.ok()) {
// 把一块虚拟地址空间交给内核做事务接收区
mVMStart = mmap(nullptr, BINDER_VM_SIZE, PROT_READ,
MAP_PRIVATE | MAP_NORESERVE, opened.value(), 0);
...
}
LOG_ALWAYS_FATAL_IF(!opened.ok(), "Binder driver '%s' could not be opened..."); // Android 上
bi
nder 不可缺
}
```

open_driver（:452）里还有一处必读：ioctl(fd, BINDER_SET_MAX_THREADS, &maxThreads)（:470-471）把 15 的上限告诉驱动——**扩池的决策方是驱动**（按并发压力发 BR_SPAWN_LOOPER），用户态只声明预算。getContextObject（:144）返回句柄 0（servicemanager）的 BpBinder，这是“找服务”的起点。

第五章 IPCThreadState：命令收发中枢

> **设计洞察 · 学习目标**
> 1. 画出transact/waitForResponse流程；2. 理解身份切换与恢复；3. 追踪BR_TRANSACTION分发

5.1 发起端：transact 与 waitForResponse

cpp frameworks/native/libs/binder/IPCThreadState.cpp:706-776（节选）

```
status_t IPCThreadState::transact(int32_t handle, uint32_t code,
const Parcel& data, Parcel* reply, uint32_t flags) {
// 讲解注释①：TF_ACCEPT_FDS——本事务允许对端"顺手"传文件描述符过来
//  （位图/FDA 的接收许可，写方无需声明、读方默认接受）。
flags |= TF_ACCEPT_FDS;
err = writeTransactionData(BC_TRANSACTION, flags, handle, code, data, nullptr); // :725 组包
if ((flags & TF_ONE_WAY) == 0) {
if (mCallRestriction != ProcessState::CallRestriction::NONE) { ... } // :733 oneway 管制检查
if (reply) {
```

```
err = waitForResponse(reply); // :751 同步：等 BR_REPLY
} else {
Parcel fakeReply;
err = waitForResponse(&fakeReply);
}
} else {
err = waitForResponse(nullptr, nullptr); // :772 oneway：收到 BR_TRANSACTION_COMPLETE 即止
}
```

mCallRestriction 是个漂亮的细节：进程可以把自己配置成“禁止同步外呼”（FATAL_IF_NOT_ONEWAY）——servicemanager 正是如此（第七章），从机制上杜绝名录服务被慢客户端的同步调用卡死。

waitForResponse（:897-990）是个命令泵：循环 talkWithDriver 读 BR_*，直到拿到本事务的结果。BR_REPLY 分支（:939-960）把驱动移交的缓冲区用 ipcSetDataReference **零拷贝**包装进 reply Parcel——freeBuffer 回调负责用完归还驱动。oneway 的终点在 BR_TRANSACTION_COMPLETE（:921）：它只表示“内核已收件”，不表示对端已执行。

5.2 服务端：executeCommand 与身份切换

cpp frameworks/native/libs/binder/IPCThreadState.cpp:1227-1273（节选）

```
case BR_TRANSACTION_SEC_CTX:
case BR_TRANSACTION: {
binder_transaction_data_secctx tr_secctx;
... // 读事务数据（缓冲区就在本进程映射区）
Parcel buffer;
buffer.ipcSetDataReference(tr.data.ptr.buffer, ...); // :1240 零拷贝包装
const pid_t origPid = mCallingPid; // :1248 保存旧身份
const uid_t origUid = mCallingUid;
...
mCallingPid = tr.sender_pid; // :1269 内核采集的调用方身份
mCallingUid = tr.sender_euid; // :1270 —— 一切权限检查的锚点
mCallingSid = reinterpret_cast<const char*>(tr_secctx.secctx); // :1271 SELinux 上下文
```

恢复与嵌套：方法执行完按 orig 值恢复身份；若服务端在处理事务时又调用第三方，transaction_stack 压栈保证返回链正确——身份与调用链都在线程上，这就是 Binder“跨进程递归”的秘密。

5.3 talkWithDriver：一次 ioctl 的行李清单

cpp frameworks/native/libs/binder/IPCThreadState.cpp:997-1100（节选）

```
status_t IPCThreadState::talkWithDriver(bool doReceive) {
binder_write_read bwr;
const bool needRead = mIn.dataCapacity() == 0 ...;
bwr.write_size = outAvail; // mOut 里积攒的 BC_* 命令
bwr.write_buffer = (uintptr_t)mOut.data();
if (needRead && doReceive) {
bwr.read_size = mIn.dataCapacity();
bwr.read_buffer = (uintptr_t)mIn.data(); // 空出的缓冲供驱动回填 BR_*
} else { bwr.read_size = 0; ... }
```

```
...
do { ioctl(mProcess->mDriverFD, BINDER_WRITE_READ, &bwr); } while (...);
```

mIn/mOut 是线程的两只手：出手（BC 命令）与接手（BR 命令）在同一 ioctl 里双向结清。**命令是批量捎带的**——几次 transat 的引用计数命令可能搭同一班车。这也是读 binder 日志的要点：一次 ioctl ≠ 一次事务。

5.4 引用计数命令：INKREFS 家族的日常

executeCommand 不只处理事务：BR_ACQUIRE/BR_INCREFS/BR_RELEASE/BR_DECREFS（对应客户端发出的 BC_* 计数命令）驱动着对象生命周期——sp<IBinder> 持有即 BC_ACQUIRE，析构即 BC_RELEASE；weak 计数走 INCREFS/DECREFS 对。当你在日志里看到成串的 IncRefs/DecRefs，那是一个 BpBinder 正在被缓存收放。**Binder 的“自动内存管理”本质是双端计数 + 驱动仲裁**，Java 层的 BinderProxy 只是把这套计数翻译成了 GC 世界的 WeakReference + 死亡通知。

3.5 BINDER_WRITE_READ ioctl 深度走读

Binder 的一切通信都经过一个 ioctl 调用——这是**最核心的系统调用**。

ioctl 的行李清单

cpp frameworks/native/libs/binder/IPCThreadState.cpp:997 起（骨架+逐行注释）

```
status_t IPCThreadState::talkWithDriver(bool doReceive) {
// 讲解注释①：binder_write_read 是 ioctl 的唯一载荷结构
struct binder_write_read bwr;
// 讲解注释②：写半区（应用→驱动）
bwr.write_size = outAvail; // mOut 中积攒的命令字节数
bwr.write_buffer = (uintptr_t)mOut.data(); // 指向 BC_* 命令流
// 典型内容：BC_TRANSACTION + binder_transaction_data（参数）
// 可能还搭 BC_INCREFS/BC_ACQUIRE（引用计数）
// 讲解注释③：读半区（驱动→应用）
if (needRead && doReceive) {
bwr.read_size = mIn.dataCapacity(); // 空出的缓冲供驱动回填
bwr.read_buffer = (uintptr_t)mIn.data();
// 驱动可能回填 BR_TRANSACTION（收到调用）
// 或 BR_REPLY（同步调用的返回）
// 或 BR_SPAWN_LOOPER（要求扩线程池）
// 或 BR_DEAD_BINDER（服务死了）
} else {
bwr.read_size = 0; // 只写不读（oneway 发送）
}
// 讲解注释④：一次 ioctl 完成双向通信
do {
ioctl(mProcess->mDriverFD, BINDER_WRITE_READ, &bwr);
} while (bwr.read_consumed > 0 && bwr.write_consumed < bwr.write_size);
// 讲解注释⑤：返回后处理
// mIn 中现在是 BR_* 命令流 → executeCommand() 分发
// mOut 中的 BC_* 已被驱动消费 → 清空
}
```

一次同步调用的 ioctl 序列

text App 调用 AMS.activityPaused() 的完整 ioctl 序列

```
App 进程（调用方）：
ioctl #1: write={BC_TRANSACTION} read={} → 发出请求，阻塞等回复
[驱动路由：写入 system_server 的 todo 队列，唤醒其 binder 线程]
[App 线程在 ioctl 中等待（内核挂起）]
system_server（服务方）：
ioctl #2: write={} read={BR_TRANSACTION} → 收到请求（之前阻塞在 ioctl 等读）
[执行业务代码]
ioctl #3: write={BC_REPLY} read={} → 发出回复
App 进程（唤醒）：
ioctl #1 返回：read={BR_REPLY} → 拿到结果
// 总结：一次同步调用 = App 1 次 ioctl（含等待）+ Server 2 次 ioctl
// 总计 3 次 ioctl（1 次阻塞等待 + 2 次数据传输）
```

设计洞察 · 逐段精读：为什么用 ioctl 而不是 read/write

① ioctl 是**全双工**（一个调用同时传写缓冲和读缓冲）——read/write 只能单向。② ioctl 可以传自定义结构体（binder_write_read），read/write 只能传字节流。③ ioctl 进入内核后可以直接操作驱动的私有数据结构（binder_proc），不需要额外的系统调用。**"用一个 ioctl 完成一次全双工事务"是 Binder 高效的核心之一**。

源码锚点 · 第五章锚点

IPCThreadState.cpp:706 transact / :725 writeTransactionData(BC_TRANSACTION)
:897 waitForResponse / :921 BR_TRANSACTION_COMPLETE / :939 BR_REPLY
:997 talkWithDriver（ioctl BINDER_WRITE_READ）
:1147 executeCommand / :1227 BR_TRANSACTION_SEC_CTX / :1269-1273 身份装载

第六章 Parcel：跨进程的类型系统

设计洞察 · 学习目标

1. 解释flat_binder_object二态；2. 说出事务即域；3. 列举常用读写API

Parcel 是双方约定的字节格式（含接口 token 校验、数据区与 offsets 表），`Parcel.cpp` 定义了全部读写。三个重点：

- **writeStrongBinder**（`:202` 构造 `flat_binder_object`）：把一个 IBinder “降维”成 {type, binder/cookie 或 handle} 结构。发送方的 `BINDER_TYPE_BINDER` 在驱动里被翻译成接收方的 `BINDER_TYPE_HANDLE`——**跨进程传递的是句柄不是指针**，收到方 `readStrongBinder` 后拿到的其实是 BpBinder 代理。

- **fd 传递**（BINDER_TYPE_FDA）：驱动为目标进程复制文件描述符表项——位图、ANR trace 管道都靠它跨进程。
- **writeInterfaceToken**：每笔事务头部写入接口名与严格模式策略，服务端 enforceInterface 校验——**协议版本对齐**的最小机制。

6.2 writeStrongBinder：对象降维现场

cpp frameworks/native/libs/binder/Parcel.cpp:195-215（节选）

```
status_t Parcel::writeStrongBinder(const sp<IBinder>& val) {
...
flat_binder_object obj;
obj.flags = acceptFds ? FLAT_BINDER_FLAG_ACCEPTS_FDS : 0;
if (val == nullptr) { obj.type = BINDER_TYPE_BINDER; obj.binder = 0; obj.cookie = 0; }
else if (val->localBinder()) {
obj.type = BINDER_TYPE_BINDER; // 本地对象：传指针+cookie
obj.binder = reinterpret_cast<uintptr_t>(val->getRefAssoc());
obj.cookie = reinterpret_cast<uintptr_t>(val.get());
} else {
obj.type = BINDER_TYPE_HANDLE; // 远端代理：传句柄
obj.handle = val->remoteBinder()->handle();
}
return finishFlattenBinder(val, obj); // 记入 offsets 表
}
```

同一个方法两种编码：**本地对象带指针、远端引用带句柄**，驱动负责翻译。offsets 表让接收方能跳着扫出所有对象槽位——Parcel 的“数据平面”与“对象平面”由此分离。常用读写速查：writeInterfaceToken/enforceInterface（协议对齐）、writeInt32/Int64/Float、writeString16、writeStrongBinder、writeFileDescriptor（FDM 数组）、writeBlob（可 zero-copy 的字节块）。**读序必须与写序严格对称**——aidl 生成代码存在的意义就是替你保证这一点。

6.1 一个易被忽视的真相：Parcel 不是序列化框架

Parcel 有生命周期约定：服务端在事务里收到的 data/reply 由 binder 缓冲区背书，**事务返回即失效**。跨事务保存 Parcel 或在异步路径上复用它是系统代码经典 bug 源。记住口诀：**事务即域（transaction-scoped）**。

第七章 servicemanager：名录与门卫

设计洞察 · 学习目标

1. 解释零线程池设计；2. 列出注册五道关卡；3. 追踪句柄0自举



图 4 servicemanager 的自举、注册五道关卡、查询与三域隔离，附“为什么这么瘦”四则

cpp frameworks/native/cmds/servicemanager/main.cpp:113-143（节选）

```
int main(int argc, char** argv) {
const char* driver = argc == 2 ? argv[1] : "/dev/binder";
sp<ProcessState> ps = ProcessState::initWithDriver(driver);
ps->setThreadPoolMaxThreadCount(0); // :125 零工作线程
ps->setCallRestriction(ProcessState::CallRestriction::FATAL_IF_NOT_ONEWAY); // :126
sp<ServiceManager> manager = sp<ServiceManager>::make(std::make_unique<Access>());
if (!manager->addService("manager", manager, ...).isOk()) { ... } // :129 自举注册
IPCThreadState::self()->setTheContextObject(manager);
ps->becomeContextManager(); // :134 ioctl(BINDER_SET_CONTEXT_MGR)
sp<Looper> looper = Looper::prepare(false);
BinderCallback::setupTo(looper); // binder fd 挂进 Looper
while (true) looper->pollAll(-1); // :141-143 单线程永续
}
```

addService（ServiceManager.cpp:295-330）的五道关卡值得背下来：app uid 拒绝（:298）→ SELinux canAdd（:302）→ 名字合法性 → VINTF 声明一致性（:317，HAL 服务必须与设备清单吻合）→ linkToDeath 挂死亡通知（:325，**服务死则自动除名**）。getService（:232）同样过 SELinux canFind——名录不只是字典，还是访问控制的第一道门。

第八章 Java 层与 AIDL：对象的两个世界

> **设计洞察 · 学习目标**
> 1. 说清Binder/BinderProxy与native对象的三层映射；2. 读懂javaObjectForIBinder的复用策略；3. 手写AIDL生成物骨架；4. 列举生成代码的十个设计决策

7.1 客户端侧：ServiceManager.java 的缓存链

Java 侧查询走 `ServiceManager.getService("activity")`：先查进程内 `sCache`（ByName map）→ 未命中经 `ServiceManagerNative.asInterface(BinderProxy(0))` 向名录发 **oneway** getBinder 请求（`ServiceManagerRaw.java` 的 getAsync 模式）→ 结果写缓存并回调。**句柄 0 是唯一天生可用的代理**——名录自身的地址被硬编码进协议，其他一切服务都从它出发。缓存的意义：名录查询从“每服务一次”降为“每进程每服务一次”；代价是服务死亡后要靠死亡通知主动失效（第九章 runtime restart 场景）。

8.1 Binder / BinderProxy 与 JNI 桥

Java 世界只有两个类需要真正理解：**Binder（本地服务端基类）** 与 **BinderProxy（远端代理）**。它们与 native 世界的缝合线在 `android_util_Binder.cpp`：

cpp frameworks/base/core/jni/android_util_Binder.cpp:749-790（节选）

```
jobject javaObjectForIBinder(JNIEnv* env, const sp<IBinder>& val) {
// 讲解注释①：checkSubclass 是"本地对象吗"的试金石——
// 本进程 JavaBBinder 携带 gBinderOffsets 印记，命中则无需代理。
if (val->checkSubclass(&gBinderOffsets)) {
// 讲解注释②：绕过一切包装直接还给调用者原 Java 对象——
// "自己进程里的服务调用自己"零开销的秘密（asInterface 本地直连）。
return static_cast<JavaBBinder*>(val.get())->object(); // :758
}
BinderProxyNativeData* nativeData = new BinderProxyNativeData();
nativeData->mOrgue = new DeathRecipientList;
nativeData->mObject = val; // :763-765 持有 BpBinder
jobject object = env->CallStaticObjectMethod(gBinderProxyOffsets.mClass,
gBinderProxyOffsets.mGetInstance, (jlong) nativeData, ...); // :767 创建/复用 BinderProxy
...
uint32_t numProxies = gNumProxies.fetch_add(1, ...); // :776 进程内代理计数
if (numProxies >= numLastWarned + PROXY_WARN_INTERVAL) {
ALOGW("Unexpectedly many live BinderProxies: %d", numProxies); // 泄漏可观测
```

三个设计点：**同 IBinder 每进程一个 BinderProxy**（getInstance 按 nativeData 复用，防止代理膨胀）；**BinderProxy 持 native 指针、native 侧持弱引用回 Java**（`JavaBBinderHolder`:462，双向生命周期由 DeathRecipientList 缝合）；**代理计数告警**——BinderProxy 泄漏（忘记 unlinkToDeath、缓存未清理）从“静默吃内存”变成日志可查。

方向反过来（`ibinderForJavaObject`:793）：Java Binder → `JavaBBinderHolder` 弱缓存的 JavaBBinder；Java BinderProxy → 直接取回 native BpBinder。`BinderProxy.transact`（`BinderProxy.java:526` → native `:1387`）就是这两张查找表的日常运转。

8.2 AIDL 生成物解剖

以框架真实的 `IActivityManager.aidl`（`core/java/android/app/`，接口声明 :89）为例，aidl 编译器生成三件套：

- **Stub**（服务端）：继承 Binder，`onTransact(code, data, reply, flags)` 里按 code 分发——每个方法一个 `TRANSACTION_x` 常量；先 `data.enforceInterface(DESCRIPTOR)` 校验协议，再逐参数 `data.readXXX()`，最后 `reply.writeXXX()`。
- **Stub.Proxy**（客户端）：持有 `mRemote`（IBinder，实为 BinderProxy），每个方法：`_data.writeInterfaceToken` + 参数 → `mRemote.transact(TRANSACTION_x, _data, _reply, 0)` → 读返回值。`asInterface(IBinder)` 按是否本进程对象决定返回本地实现还是 Proxy。
- **Parcelable 参数**：aidl 定义的 `in/out/inout` 决定数据流向——`out` 参数在服务端 new 空对象、由 reply 回填。

手写一个 MiniBinder 服务（不复用 SystemService）只需要：定义 aidl → 服务端 **new Stub() { 实现 }** 并 `ServiceManager.addService("mysvc", this)` → 客户端 `IMySvc.Stub.asInterface(ServiceManager.getService("mysvc"))`。三行代码背后，正是本册第二到第八章的全部机器。

8.3 Stub/Proxy 生成物还原（典型结构）

aidl 编译产物可用约三十行还原其骨架（依 aidl 语义手写，标注与源码对应关系）：

java IActivityManager 生成物的典型结构（依 aidl 语义还原）

```
public interface IActivityManager extends android.os.IInterface {
public static abstract class Stub extends android.os.Binder
implements IActivityManager {
static final int TRANSACTION_activityPaused = FIRST_CALL_TRANSACTION + n;
public boolean onTransact(int code, Parcel data, Parcel reply, int flags) {
data.enforceInterface(descriptor); // 协议对齐
switch (code) {
case TRANSACTION_activityPaused: {
IBinder b = data.readStrongBinder(); // 读参数（按序！）
long token = data.readLong();
activityPaused(b, token); // 调真身
reply.writeNoException(); // 回执
return true;
} ...
}
}
private static class Proxy implements IActivityManager {
private IBinder mRemote; // 实为 BinderProxy
public void activityPaused(IBinder token, long pausedAt) {
Parcel _data = Parcel.obtain(), _reply = Parcel.obtain();
_data.writeInterfaceToken(descriptor);
_data.writeStrongBinder(token); _data.writeLong(pausedAt);
mRemote.transact(TRANSACTION_activityPaused, _data, _reply, 0);
_reply.readException(); // 同步等结果
}
```

```
}
public static IActivityManager asInterface(IBinder obj) {
if (obj instanceof IActivityManager) return (IActivityManager) obj; // 本地直连
return new Proxy(obj); // 远端包代理
}
}
}
```

读真实生成代码（out/soong/.intermediates/.../IActivityManager.java，本树 out/ 若有构建产物可直接翻）会发现更多工程细节：_data.recycle() 归还 Parcel 池、异常经 writeException 编码回传、oneway 方法没有 _reply。**手写一遍这个骨架**是检验你是否真懂本册前八章的最佳练习。

9.3 Binder 性能调优实战（教科书级教程）

何时需要调优

text Binder 性能问题的典型信号

```
① dumpsys binder_calls_stats 某接口 >1ms 均值
② perfetto binder_track 轨有持续长条
③ ANR traces 显示"waiting to send binder"或"binder thread full"
④ logcat 频繁 "binder: ... transaction failed"
```

五步调优法

bash

```
# 步骤 1：定位热点接口
adb shell dumpsys binder_calls_stats --enable
# ... 运行场景 ...
adb shell dumpsys binder_calls_stats | grep -E "mean_ms|count"
# 找到 mean_ms 最高的接口
# 步骤 2：抓取事务级 trace
# perfetto 配置开启 binder_track
adb shell perfetto -c /data/misc/perfetto-configs/binder_trace.txt -o
/data/misc/perfetto-traces/tra
ce
# 步骤 3：分析事务链
# 在 trace 中找到目标接口 → 看唤醒-执行-回复三段耗时
# 唤醒慢 = 目标进程 busy（不是 Binder 本身慢）
# 执行慢 = 业务代码慢
# 回复慢 = 数据序列化大
# 步骤 4：定向优化
# 场景 A：调用太频繁 → 批量化接口（一次传一批 vs N 次单传）
# 场景 B：数据太大 → 改用 fd + SharedMemory（如位图）
# 场景 C：目标线程池满 → 增大 maxThreads 或减少同步调用
# 步骤 5：复验
# 重新跑 dumpsys binder_calls_stats 对比优化前后
```

常见优化场景

场景	原因	方案	效果
App 高频查询 AMS	每帧一次 getService	SystemServiceRegistry 缓存	消除 95% 调用
大位图跨进程	Parcel 序列化慢	fd + SharedMemory	10x 加速
system_server 线程满	31 个线程全忙	减少嵌套同步调用	消除阻塞
oneway 塞满 async space	高频 oneway 大数据	分片或降频	避免 FAILED_REPLY

> **设计洞察 · 逐段精读：Binder 慢 ≠ Binder 的错**
> 十跳中真正可控的是"业务执行"（第八跳）和"数据序列化"（第二跳和第九跳）。**Binder 协议本身（驱动+IPCThreadState）极少是瓶颈**——它只占 10-50μs。大部分"Binder 慢"其实是服务端业务慢或客户端序列化慢。先用 perfetto 定位是哪一段慢，再定向优化。

8.4 AIDL 生成物完整解剖（教科书级逐步分析）

AIDL 编译器从 .aidl 文件自动生成 Java/C++ 绑定代码。以下以 `IActivityManager.aidl` 的一个方法为例，完整解剖生成物。

输入：.aidl 文件

java frameworks/base/core/java/android/app/IActivityManager.aidl（节选）

```
interface IActivityManager {
// 讲解注释①：接口声明
void activityPaused(in IBinder token, long pausedAt);
// 参数修饰符：in=只传过去 / out=只传回来 / inout=双向
}
```

输出1：Stub（服务端存根）

java AIDL 编译器生成的 Stub（骨架+逐行注释）

```
public static abstract class Stub extends android.os.Binder
implements IActivityManager {
// 讲解注释②：DESCRIPTOR 是接口的唯一标识
private static final String DESCRIPTOR = "android.app.IActivityManager";
// 讲解注释③：每个方法分配一个事务码（从 FIRST_CALL_TRANSACTION 递增）
static final int TRANSACTION_activityPaused = (android.os.IBinder.FIRST_CALL_TRANSACTION +
0);
// 讲解注释④：asInterface——本地直连或远程代理的分水岭
public static IActivityManager asInterface(android.os.IBinder obj) {
if (obj == null) return null;
IActivityManager in = (IActivityManager)obj.queryLocalInterface(DESCRIPTOR);
if (in != null) return in; // 本进程 → 直接返回本地实现
return new Stub.Proxy(obj); // 远程 → 包装代理
}
// 讲解注释⑤：onTransact——服务端的"路由器"
@Override
public boolean onTransact(int code, Parcel data, Parcel reply, int flags) {
switch (code) {
```

```
case TRANSACTION_activityPaused: {
// 讲解注释⑥：协议校验（防止版本不匹配）
data.enforceInterface(DESCRIPTOR);
// 讲解注释⑦：按序读取参数（与 Proxy 的写入序严格对称）
android.os.IBinder _arg0 = data.readStrongBinder();
long _arg1 = data.readLong();
// 讲解注释⑧：调用真实实现（AMS 的方法）
this.activityPaused(_arg0, _arg1);
// 讲解注释⑨：写回执
reply.writeNoException();
return true;
}
}
return super.onTransact(code, data, reply, flags);
}
}
```

输出2：Stub.Proxy（客户端代理）

java AIDL 编译器生成的 Proxy（骨架+逐行注释）

```
private static class Proxy implements IActivityManager {
private android.os.IBinder mRemote; // 实际是 BinderProxy
@Override
public void activityPaused(android.os.IBinder token, long pausedAt)
throws android.os.RemoteException {
// 讲解注释⑩：准备两个 Parcel
android.os.Parcel _data = android.os.Parcel.obtain();
android.os.Parcel _reply = android.os.Parcel.obtain();
try {
// 讲解注释⑪：写入协议标识+参数（与 Stub 的读取序严格对称）
_data.writeInterfaceToken(DESCRIPTOR);
_data.writeStrongBinder(token);
_data.writeLong(pausedAt);
// 讲解注释⑫：发起 Binder 调用（书3 十跳的第一跳）
mRemote.transact(Stub.TRANSACTION_activityPaused, _data, _reply, 0);
// 讲解注释⑬：读取异常+返回值
_reply.readException();
} finally {
// 讲解注释⑭：归还 Parcel（池化复用，不产生 GC）
_reply.recycle();
_data.recycle();
}
}
}
```

生成物中隐藏的 10 个设计决策

设计	生成代码中的体现	为什么
DESCRIPTOR 字符串	writeInterfaceToken/enforceInterface	版本兼容：新旧版本接口共存
FIRST_CALL_TRANSACTION 递增码	TRANSACTION_activityPaused = 0	一个 Binder 对象可以承载多个接口

设计	生成代码中的体现	为什么
asInterface 本地判断	queryLocalInterface	同进程直连省一次 Binder 开销
Parcel 池化	obtain/recycle	避免高频 GC
in/out/inout	生成代码中读写顺序不同	数据方向=写入侧写、读取侧读
异常编码	writeException/readException	跨进程异常不能直接 throw
线程安全	Proxy 无锁	每个线程独立 IPCThreadState
oneway 编译	flags 参数=ThreadFlags.FLAG_ONEWAY	编译期确定是否阻塞
默认实现	default 方法	新方法不破坏老客户端
raw/bool 转 int	writeInt/readInt	Parcel 只支持基础类型

设计洞察 · 逐段精读：为什么要自动生成

同一套模式（token→参数→transact→异常→回收）重复 140+ 次（IActivityManager 的方法数）——**手写必出错、自动生成保证一致性**。这就是"代码生成器"的核心价值：把可模式化的东西交给机器。理解了 AIDL 生成物的结构，你就掌握了 Android 接口定义的全部秘密。

源码锚点 · 第八章锚点

Binder.java:330/340 getCallingPid/getCallingUid（native 直读 IPCThreadState）
Binder.java:397 clearCallingIdentity / restoreCallingIdentity
BinderProxy.java:526/584/614 transact → transactNative
android_util_Binder.cpp:749 javaObjectForIBinder / :793 ibinderForJavaObject / :1387 transact JNI
IActivityManager.aidl:89 interface 声明（39KB，140+ 方法——AMS 面向全机的合同书）

第九章 端到端全链路：把图走成路

设计洞察 · 学习目标

1. 不看图复述十跳；2. 说出服务缓存与句柄失效的联动；3. 解释线程选择与扩池时机；4. 理解回复为何不新建缓冲

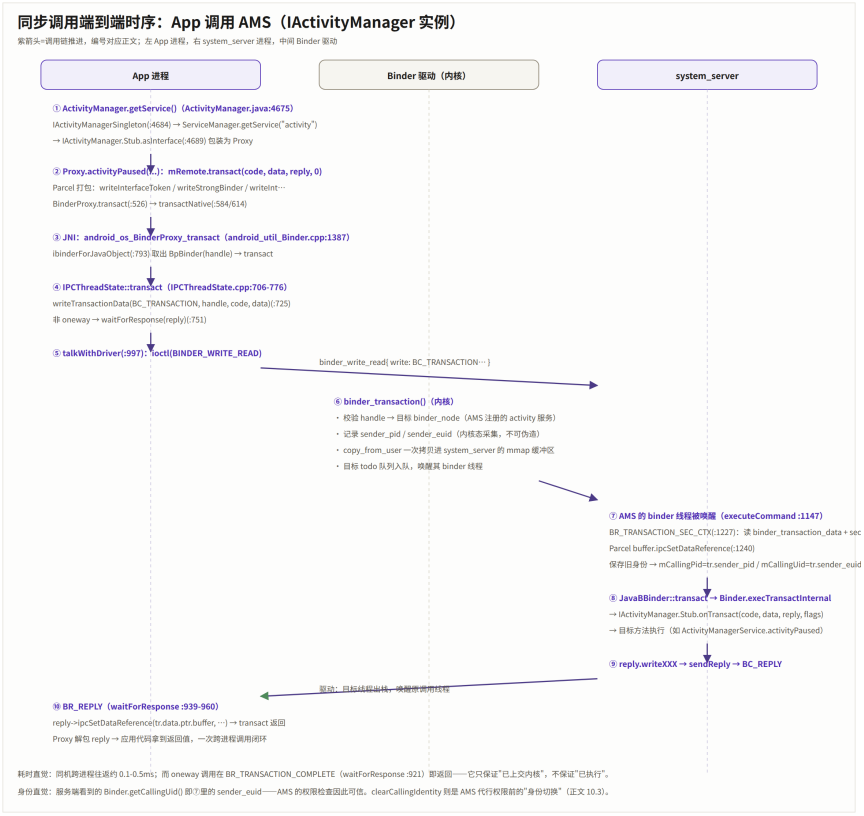


图 5 同步调用端到端时序：App 调用 AMS 的十跳全景（含驱动的采集与拷贝、服务端身份装载）

配合时序图逐跳复述（正文不再重复细节，只补三处“图上放不下”的注释）：

- **第①跳**的 `IActivityManagerSingleton`（`ActivityManager.java:4684-4690`）是进程级单例缓存——`getService` 只在首次发生，此后直连句柄。**服务重启后句柄失效**靠 BinderProxy 死亡通知触发缓存刷新，这也是“system_server 重启（runtime restart）后 App 要重新拿服务”的机制层解释。
- **第⑥跳**驱动里除了拷贝与排队，还要做**目标线程选择**：优先找空闲 binder 线程，全忙则看是否超过线程预算决定扩池（`BR_SPAWN_LOOPER`）或排队。因此“服务端线程打满”不是异常，是供给不足的常态信号。
- **第⑨跳**的 BC_REPLY 走的是与第⑥跳对称的路径，但**不新建缓冲**：回复复用调用方事务的预留空间，往返的内存账本始终只有一本。

第十章 线程池与高级机制

> **设计洞察 · 学习目标**
>
> 1. 画出16线程模型；2. 解释oneway三误区；3. 追踪死亡通知链



图 6 Binder 线程池：来源（主线程+驱动扩池）、一生（双面性）、三条铁律与冻结/泛洪新语义

10.1 死亡通知：优雅地处理“对方没了”

`linkToDeath(DeathRecipient)` 走 `BC_REQUEST_DEATH_NOTIFICATION`，目标 node 销毁时驱动回推 `BR_DEAD_BINDER`。框架里最著名的用户是 AMS：每个应用进程的 `ProcessRecord` 都挂 `AppDeathRecipient`（`ProcessList.java:5056-5058` 创建、`:1648/4937` 解绑）——**App 被杀后 AMS 收到回调清理组件与窗口**，用户感知的“应用重启”从这里开始。servicemanager 的自动除名（第七章）是同一机制的另一个客户。

10.2 身份切换：clearCallingIdentity 的用途

服务端方法里 `Binder.getCallingUid()` 是调用方身份；但当 AMS **代表系统**去做下一步动作（如启动 Activity）时，必须先 `clearCallingIdentity()`（`Binder.java:397`）切回本进程身份，事毕 `restoreCallingIdentity` 恢复。忘切 = 把调用方权限放大给系统（提权漏洞）；忘恢复 = 后续调用者身份错乱。**所有做系统服务的同学都要把这组 API 写进肌肉记忆**。

10.2.1 身份切换代码示例

java 服务端典型模式（AMS 各方法俯拾皆是）

```
public void startInstrumentation(...) {
final int callingUid = Binder.getCallingUid(); // 调用方身份（驱动背书）
enforceCallingPermission(...) // 用调用方身份做检查
final long ident = Binder.clearCallingIdentity(); // 切换为 system_server 本身
try {
mContext.startActivityAsUser(..., Process.myUserHandle()); // 代行系统权限
} finally {
```

```
Binder.restoreCallingIdentity(ident); // 必须恢复：后续事务身份不可污染
}
}
```

规则化表述：**检查用调用方身份，动作用系统身份，边界必须 try/finally**。这两组 API 的不对称性（clear 返回 token、restore 消费 token）就是为强制成对使用而设计。

10.3 oneway 的真实语义

三个常被误解的点：oneway **不保证对端已执行**（只到 BR_TRANSACTION_COMPLETE）；对**同一目标进程**的 oneway 在驱动内保序（单 async 队列），跨进程自然可能乱序；async 空间（映射区约一半）是独立预算，塞满则 BR_FAILED_REPLY——大位图走 oneway 会翻车的根源。Android 13 还加了泛洪检测 BR_ONEWAY_SPAM_SUSPECT（waitForResponse:915）：异常高频的 oneway 发送者会被点名进日志。

10.4 冻结语义（Android 12+）

缓存应用被 cgroup freezer 冻结时，发往它的**同步**事务立刻 BR_FROZEN_REPLY（:931）而非挂起等超时——把“冻结进程的 binder 行为”从模糊变成确定，是后台冻结功能能落地的协议前提。

10.5 libbinder_ndk 与稳定性等级

现代 C++ 服务（包括 AIDL HAL）多走 **libbinder_ndk**（frameworks/native/libs/binder/ndk）：AIBinder/AStatus/AIBinder_setInheritRtl? 这套 C API 把 libbinder 包成可由 NDK 稳定暴露的薄层，aidl 编译器可为同一接口产出 backend = java/cpp/ndk/rust 四份绑定——**接口一次定义，四种语言的客户端与服务端互通**。配套的**稳定性等级**（Stability / VINTF）：none（进程内）→ vendor（vendor 分区接口需 VINTF 登记版本）→ system（可跨分区）→ vintf（最高承诺）。等级决定接口能否演化、跨哪些分区可用——给系统加服务时，选错等级比写错代码更难补救。

10.6 线程栈实勘

text debuggerd -b <system_server_pid>（binder 线程栈节选，示意）

```
"Binder:1234_5" ... sys_waitid / ioctl(/dev/binder) ← 空闲：阻塞在 talkWithDriver
"Binder:1234_9" ... IPCThreadState::transact ← 正外呼他人（出站）
→ android_os_BinderProxy_transact
→ com.android.server.am.XXX.method ← Java 业务栈
"Binder:1234_12" ... BBinder::transat ← 入站：正处理别人的事务
→ IActivityManager.Stub.onTransact
```

空闲线程沉在 ioctl、出站线程停在 transact、入站线程挂在业务——三类栈型一眼可辨。ANR/卡顿分析时先数 binder 线程构成，再定位慢在哪一型，是系统性能的基本功。

第十一章 设计决策十二问

设计洞察 · 学习目标

1. 用一句话回答每个"为什么"；2. 把每问对应回正文章节；3. 尝试举出违反该决策的反例后果；4. 迁移：为你的新IPC设计回答同样的问题

- **为什么 mmap 接收区而不是共享内存池？** 每进程私有 1MB 线性区，配额隔离、免全局锁；PROT_READ 只读防篡改（第一章）。
- **为什么 15+1 线程上限？** 防 binder 线程吃光进程内存；供给制（驱动 SPAWN_LOOPER）按需扩容（第十章）。
- **为什么身份在驱动采集？** 用户态任何“自报身份”都可伪造；`sender_euid` 是权限体系的信任根（第五章）。
- **为什么 oneway 有独立空间？** 无限 oneway 会淹没接收方；async 半区配额把洪泛伤害封顶（第三章/十章）。
- **为什么每进程一个 BinderProxy？** 防代理膨胀 + 泄漏可观测（第八章）。
- **为什么 servicemanager 零线程池、仅 oneway？** 名录必须极简极稳，绝不能被客户端行为牵连（第七章）。
- **为什么句柄 0 自举？** “找服务”本身也是服务——驱动把名录固定在特殊句柄，打破循环依赖（第七章）。
- **为什么注册即 linkToDeath？** 名录与进程生命周期同步，杜绝“名录里的僵尸服务”（第七章）。
- **为什么同步用线程栈而非全局表？** 栈天然支持跨进程递归调用链，无需回滚逻辑（第三章）。
- **为什么 AIDL 而非手写 Parcel？** 协议生成保证两端一致（enforceInterface/token）；稳定 AIDL 还给跨进程版本演化立规矩（第八章）。
- **为什么 system_server 31 线程且禁同步外呼？** 高并发预算 + 防优先级反转，框架进程的“特权”其实是“责任”（第十章）。
- **为什么事务缓冲区是事务即域？** 1MB 区是稀缺资源，驱动靠“用完即还”维持周转；Parcel 跨事务保留 = 机器里塞沙子（第六章）。

第十二章 深水区：Binder 观测武器库与 RPC 后端

设计洞察 · 学习目标

1. 分层说出三层观测手段；2. 组合三招定位一次卡顿；3. 理解RPC后端"协议与传输分离"的野心；4. 知道debugfs与binder_calls_stats各自能读到什么

12.1 三层观测（从粗到细）

层	手段	能看到什么
服务级	dumpsys binder_calls_stats（需开启）	每接口调用计数/耗时/拒绝码分布（BinderCallsStatsService，书1 附录B 曾列）
进程级	dumpsys binder <pid> / cat /d/binder/*	proc/thread 状态、事务栈、pending、缓冲水位（/d/binder/stats、transactions、proc/…）
事务级	perfetto 的 binder_track	每次 BC_TRANSACTION→BR_REPLY 的时序轨（含目标服务/代码号），配合调度轨看唤醒等待

排障组合拳：卡顿先 `binder_calls_stats` 找热点接口 → `/d/binder/transactions` 看谁压栈未弹 → perfetto 精确到一次调用的唤醒-执行-回复三段。**事务栈里有别人=嵌套调用现场**（书3 §3.2 的 transaction_stack 实体就在这个 debugfs 文件里）。

12.2 RPC 后端（Android 13 的实验分支）

本树 `frameworks/native/libs/binder/RpcState.cpp` 提示了 Binder 的"第二人生"：**RPC backend**（`binder_rpc`）把事务协议搬到任意传输（Unix socket/TLS/USB）上运行，面向多设备场景（手机↔车机↔穿戴）——协议层 Binder 化、传输层可插拔。Parcel 因此有了"for RPC"分支（本册 transact :710 的 isForRpc 断言来源）。读码价值：看懂"Binder 是协议不是设备"的架构野心，也解释了 13 里 Parcel/状态码的一批防御性改动。

> **源码锚点 · 本章锚点**
> frameworks/native/libs/binder/Static.cpp：全进程 binder 统计与死亡通知簿
IPCThreadState.cpp 的 dump/命令字符串表（:75-97）
/d/binder/（debugfs，userdebug 可读）：stats/transactions/proc/<pid>/
RpcState.cpp：RPC 后端的事务簿记

附录 A 观测实战

目标	手段	对应机制
看服务名录	adb shell service list	servicemanager mNameToService
看某服务实现	dumpsys <名>（每个服务实现 dump）	服务端 onTransact 之外的 dump 通道
看线程池状态	debuggerd -b <pid> 看 binder 线程栈	joinThreadPool 循环
看调用统计	dumpsys binder_calls_stats（需开启）	BinderCallsStatsService（书1表）

目标	手段	对应机制
验证身份切换	dumpsys activity 里 uid 记录	getCallingUid/clearCallingIdentity
复现 oneway 限制	大 oneway 事务 → FAILED_TRANSACTION	async space 配额
验证死亡通知	kill 某服务进程，观察名录自动除名	linkToDeath + BR_DEAD_BINDER

附录 B 三册联动：一条完整的因果链

第二册结尾 system_server 已就绪、Home 已启动。本册补完了此后每一天的运转逻辑：App 调用系统服务（本册第九章十跳）→ AMS 跨进程反向控制 App（IApplicationThread，同一机制的对称方向）→ 应用死亡由 AppDeathRecipient 清理（本册十章）→ 系统重启时名录与缓存全部重来（第二册启动链 + 本册句柄失效）。至此，“从开机到点击图标打开 App”的全栈白盒在你手中合拢。

附录 C 自检题

- 说出一次同步 Binder 调用的十跳（不看图）。
- `getCallingUid` 的值为什么可信？数据从哪来、经过哪几层？
- oneway 调用在什么时刻返回？什么情况下失败？失败码是什么？
- BinderProxy 与 BpBinder 的关系？为什么需要两层？
- AMS 为什么必须在某些操作前 clearCallingIdentity？举一个不这么做会出提权漏洞的场景。

附录 D 术语速查

术语	解释
Binder 驱动	/dev/binder 背后的内核模块，协议仲裁与一次拷贝的实现者
BC_* / BR_*	应用→驱动 / 驱动→应用 的命令前缀
binder_proc/thread/node/ref/buffer	驱动侧五类核心对象
transaction_stack	线程上的同步调用链栈，支撑跨进程递归
async_space / async_todo	oneway 专属配额与队列（保序）
BINDER_SET_CONTEXT_MGR	成为名录（句柄 0）的 ioctl
ProcessState / IPCThreadState	每进程 / 每线程 的 libbinder 单例
BBinder / BpBinder	native 服务端基类 / 远端代理（按句柄）

术语	解释
JavaBBinder / BinderProxy	Java Binder 的 native 壳 / native BpBinder 的 Java 马甲
Parcel（事务即域）	跨进程字节容器，生命周期随事务
flat_binder_object	Parcel 里"对象槽"的线格式（本地指针/远端句柄二态）
oneway	单向事务：COMPLETE 即返回，同目标保序，不保证执行
死亡通知	linkToDeath + BR_DEAD_BINDER 的对象死亡回调机制
Stability / VINTF	接口稳定性承诺等级与设备兼容矩阵
libbinder_ndk	面向 NDK/Rust 等的 C 稳定封装（AIBinder 家族）
三域	binder / hwbinder / vndbinder 的框架/硬件/厂商隔离

附录 E 动手实验（一天量）

实验一“名录巡礼”：`adb shell service list` 找到 activity/power/window；`adb shell service call activity 1`（code 需对照生成物，多半报错）——观察错误类型，理解 TRANSACTION code 的含义与 enforceInterface 的拦截。

实验二“线程构成”：对 system_server 连续抓三次 `debuggerd -b`（或 `cat /proc/<pid>/task/*/comm | grep Binder`），统计 binder 线程数变化，验证“31+1”预算与供给制扩池。

实验三“MiniBinder”：写一个 20 行的 App 内 demo——定义 `interface IDemo { int add(int a,int b); }` 的 aidl，Service 端实现并 addService（系统签名不可用时可用 bindService 路径替代名录），客户端调用。对照 8.3 骨架找到你代码里的 enforceInterface/transat 位置。

实验四“oneway 边界”：同一 aidl 方法加/去 oneway 各压测一百次，对比耗时分布；再传一个 2MB byte[]（oneway），复现 async 配额耗尽的 FAILED_TRANSACTION。

实验五“死亡通知”：bindService 后 kill 服务进程，客户端 DeathRecipient 回调链上打点，画出从进程死亡到你收到回调的时间线（对照 10.1）。

第十三章 Parcel 线格式：字节级走读

> **设计洞察 · 学习目标**
> 1. 画出mData与mObjects双平面布局；2. 解释4字节对齐与pad mask；3. 逐行追踪writeStrongBinder的编码路径；4. 说清事务头三件套与稳定性尾注的作用

第六章告诉你 Parcel "是什么"，本章把它拆到字节。一次 AIDL 调用的参数在进入 `writeTransactionData`（IPCThreadState.cpp:1104，十跳第二跳的最后一站）之前，究竟以什么字

节序列躺在缓冲区里？这一层是协议的**线格式**（wire format）：它与第八章的 AIDL 生成物首尾相接（Proxy 的写 API 调用序就是本章的函数调用序），与第三章的驱动翻译共用同一种语言（`flat_binder_object` 是用户态与驱动共同解析的结构）。读懂本章，你就能在 hexdump 一份真实 Parcel 时逐字节点名。

13.1 双平面：mData 与 mObjects

Parcel 在内存里维护两张平面（`Parcel.h:1247-1253`）：`mData/mDataSize/mDataCapacity/mDataPos` 描述**数据平面**（普通字节，线性增长），`mObjects/mObjectsSize/mObjectsCapacity` 描述**对象平面**——一个偏移量数组，记录"数据平面里第几个字节开始是一个需要驱动翻译的对象槽"。写作时 `writeObject` 把偏移量登记进 mObjects；ioctl 时 `writeTransactionData` 把两张平面分别填进 `binder_transaction_data` 的 `data.ptr.buffer/offsets` 两个指针（IPCThreadState.cpp:1122-1124）——驱动据此**跳着扫**：只看对象槽，不解析全部数据。为什么分离？若把对象翻译混进字节流，驱动就得理解每种类型的二进制布局；分离后驱动只认识一种 24 字节的结构（flat_binder_object），数据平面全程不透明。这是"控制平面与数据平面分离"在 IPC 协议里的教科书实例。

cpp frameworks/native/libs/binder/Parcel.cpp:871-910（writeInplace：对齐与补零现场）

```
void* Parcel::writeInplace(size_t len)
{
const size_t padded = pad_size(len); // 讲解注释①：pad_size(:68) 包装 :66 的
// PAD_SIZE_UNSAFE=((s)+3)&~3——一切写入
// 都向上取整到 4 字节——线格式无"半个字"
if (mDataPos+padded < mDataPos) {
return nullptr; // 讲解注释②：size_t 回绕检查——超大 len 的整数溢出防御
}
if ((mDataPos+padded) <= mDataCapacity) {
restart_write:
uint8_t* const data = mData+mDataPos; // 讲解注释③：数据平面当前写指针，mDataPos
是读者与写者共享的"游标"
// Need to pad at end?
if (padded != len) {
#if BYTE_ORDER == LITTLE_ENDIAN
static const uint32_t mask[4] = {
0x00000000, 0x00ffffff, 0x0000ffff, 0x000000ff
};
#endif
*reinterpret_cast<uint32_t*>(data+padded-4) &= mask[padded-len];
// 讲解注释④：pad mask——len=3 时 padded=4，把最后一字的最高字节清零。
// 补的不是 0x00 字节流而是"掩码清零"，保证任何读者读 4 字节都是合法 uint32。
// BIG_ENDIAN 分支(:888)用另一张 mask——线格式同时承诺大端机器可读
}
finishWrite(padded); // 讲解注释⑤：mDataPos += padded (:811)
// finishWrite 还负责 mDataSize 水位线推进
return data;
}
status_t err = growData(padded); // 讲解注释⑥：容量不足则扩容（新容量≈1.5 倍），回跳重写
if (err == NO_ERROR) goto restart_write;
return nullptr;
```

```
}
```

13.2 一笔真实事务的字节布局

把 `writeInterfaceToken("com.example.IMySvc")` + `writeInt32(42)` + `writeStrongBinder(localBinder)` 的产物画出来（系统分区进程、x86 小端机）：

text 一笔最小事务的 Parcel 数据平面字节布局（偏移为示意）

```
偏移 内容 写入者
——————————————————————————————————————————————
0x00 strict mode policy | penalty gather writeInterfaceToken :691
0x04 work source uid（或 kUnsetWorkSource） :693 传播调用溯源
0x08 kHeader = 'S','Y','S','T' :695 分区身份徽章（见 15.4）
0x0C utf16 长度 N writeString16 :925
0x10 接口名字符串 N×2 字节 + 终止符 :931（writeInplace 补齐到 4）
0x.. int32 = 42 writeInt32 :1052 → writeAligned
0x.. flat_binder_object（24 字节） writeObject :1402
├ hdr.type = BINDER_TYPE_BINDER 本地对象（远端则为 HANDLE+句柄）
├ flags = ACCEPTS_FDS|schedBits 是否收 fd/调度策略继承
├ binder = weakrefs 指针 驱动翻译前的"本地真身"
└ cookie = BBinder 指针 驱动原样回传给本进程
0x.. int32 = 稳定性等级 repr finishFlattenBinder :154
——————————————————————————————————————————————
对象平面 mObjects = [每个 flat_binder_object/bsFD 在数据平面的偏移]
// 讲解注释①：头部 12 字节是"协议对齐三件套"——严格模式策略、WorkSource、分区徽章
// 讲解注释②：接口 token 是变长 utf16（长度前缀+内容+终止符），为对齐补零
// 讲解注释③：flat_binder_object 的 cookie 对发送方是 BBinder 指针，
// 经驱动翻译到接收方变成 BINDER_TYPE_HANDLE，cookie 清零——见第六章
```

事务头三件套值得多看一眼。strict mode policy（:691）把调用方的严格模式惩罚策略随事务传播——服务端磁盘读写违规能算到调用方头上；WorkSource（:693）让"谁触发了这次电量消耗"可归因（后台Job 调用时尤其重要）；kHeader（:675-679）是编译期烙进 libbinder 的分区徽章——__ANDROID_VNDK__ 编译出 `'V','N','D','R'`，recovery 编译出 `'R','E','C','O'`，系统侧是 `'S','Y','S','T'`（B_PACK_CHARS 把 4 个字符拼成一个 int32）。服务端可据此识别"这笔调用来自哪个世界"。

13.3 writeStrongBinder 全链：降维的五步

第六章 6.2 给了骨架，本章补全从调用到落盘的完整五步，重点在新发现的三处：

cpp frameworks/native/libs/binder/Parcel.cpp:180-258 + :1402-1427 + :154-163（拼合走读）

```
status_t Parcel::flattenBinder(const sp<IBinder>& binder) {
BBinder* local = nullptr;
if (binder) local = binder->localBinder();
if (local) local->setParceled(); // 讲解注释①：本地对象第一次被跨进程传递时标记
//  （配合 Stability 的"首次定型"，见 15.4）
...
if (binder != nullptr) {
if (!local) { // 讲解注释②：远端引用（BpBinder）——原样转发句柄
const int32_t handle = proxy ? proxy->getPrivateAccessor().binderHandle() : 0;
obj.hdr.type = BINDER_TYPE_HANDLE; // 服务端转发别人家的服务时走这支
```

```
obj.handle = handle;
obj.cookie = 0; // 句柄语义归驱动所有，cookie 置空
} else { // 讲解注释③：本地对象——传真身
obj.flags = FLAT_BINDER_FLAG_ACCEPTS_FDS; // 默认允许对端回传 fd
if (local->isRequestingSid())
obj.flags |= FLAT_BINDER_FLAG_TXN_SECURITY_CTX; // 要求驱动附带 SELinux 上下文
if (local->isInheritRt())
obj.flags |= FLAT_BINDER_FLAG_INHERIT_RT; // 实时优先级继承（13 引入）
obj.hdr.type = BINDER_TYPE_BINDER;
obj.binder = reinterpret_cast<uintptr_t>(local->getWeakRefs()); // 弱引用表地址
obj.cookie = reinterpret_cast<uintptr_t>(local); // 本体——驱动会原样带回
}
}
obj.flags |= schedBits; // 讲解注释④：调度策略位图（schedPolicyMask :195）
// 本地对象可覆盖：getMinSchedulerPolicy
status_t status = writeObject(obj, false); // 讲解注释⑤：真正落盘+登记（见下）
return finishFlattenBinder(binder); // 讲解注释⑥：尾注 int32 稳定性等级
}
status_t Parcel::writeObject(const flat_binder_object& val, bool nullMetaData) {
...
restart_write:
*reinterpret_cast<flat_binder_object*>(mData+mDataPos) = val; // 讲解注释⑦：24 字节整体 memcpy
if (val.hdr.type == BINDER_TYPE_FD) { // 讲解注释⑧：fd 类型要过 mAllowFds 闸门
if (!mAllowFds) return FDS_NOT_ALLOWED; //  （writeFileDescriptor 被 android.os.
} // Parcel 政策禁用时立刻失败）
if (nullMetaData || val.binder != 0) {
mObjects[mObjectsSize] = mDataPos; // 讲解注释⑨：对象平面登记——只记偏移
acquire_object(ProcessState::self(), val, this); // 本进程先加引用，防"还没发就析构"
mObjectsSize++;
}
return finishWrite(sizeof(flat_binder_object));
}
status_t Parcel::finishFlattenBinder(const sp<IBinder>& binder) {
internal::Stability::tryMarkCompilationUnit(binder.get()); // 讲解注释⑩：首次定型稳定性
int16_t rep = internal::Stability::getRepr(binder.get());
return writeInt32(rep); // 讲解注释⑪：4 字节尾注随对象一起飞
} // 读侧 finishUnflattenBinder(:161) 校验
```

设计洞察 · 逐段精读：为什么每个对象都带 4 字节稳定性尾注

flat_binder_object 只负责"这是谁"，不负责"这接口承诺多稳"。跨进程收到一个 binder 引用时，接收方要决定：能把它存进缓存慢慢用吗？能转发给更低分区的进程吗？答案就在紧跟其后的 int32 尾注里（UNDECLARED/VENDOR/SYSTEM/VINTF，见 15.4）。尾注由发送方 finishFlattenBinder 写入、接收方 finishUnflattenBinder 读出并经 Stability::setRepr 定型（Parcel.cpp:161-179）——一次平坦化、一次非平坦化，对称成对，这正是"对象能力自带元数据"的设计：**引用本身携带它的信任级别**，无需额外查询。

三个此前容易忽略的点收尾：其一，`acquire_object`（:101）在写入瞬间就给对象加引用——Parcel 可能先攒着稍后发，这期间对象不能析构；发不出去时 `freeData` 走 `release_object` 对称回收。其二，`BINDER_TYPE_FD` 的 `mAllowFds` 闸门（:1412）就是 Java 层 **Parcel 政策** 的底层开关——`android.os.StrictMode` 禁止 fd 泄漏时把这道闸焊死。其三，RPC 后端（12.2）不走 flat_binder_object：`flattenBinder` 开头的 `isForRpc()` 分支（:212-228）改写为 **int32 non-null 标**

记 + uint64 地址——同一个语义，两套线格式，因为 RPC 的对象簿记在 RpcState 不在驱动。对照 8.4 的生成物，你会发现 Proxy 写参数的顺序就是 13.2 表格从上到下的顺序——**AIDL 是本章的语法糖**。

源码锚点 · 第十三章锚点

Parcel.cpp:66 PAD_SIZE_UNSAFE / :811 finishWrite / :871 writeInplace（pad mask）
:675-679 kHeader 分区徽章 / :690-697 writeInterfaceToken 三件套
:200-258 flattenBinder / :1402 writeObject（mObjects 登记+acquire_object）
:154 finishFlattenBinder（稳定性尾注）/ Parcel.h:1247-1253 双平面成员
UAPI binder.h:125 flat_binder_object（hdr/flags/union/cookie 共 24 字节）

第十四章 错误路径：DEAD、FAILED 与状态码通道

设计洞察 · 学习目标

1. 区分错误的三种诞生地；2. 追踪TF_STATUS_CODE状态通道；3. 说出FAILED_TRANSACTION的双重人格；4. 解释oneway失败的静默性

第九章走的是晴天的十跳，本章专讲下雨：调用失败时每一跳发生什么、错误码在哪一层诞生、又如何变成你 catch 到的异常。链路位置：十跳的①②（发送端组包失败）、⑤（驱动拒绝）、⑩（服务端异常回传）——同一件事"调用没成"，其实有**三种诞生地**，排查时的入口完全不同。第五章 5.1 已给出 `waitForResponse` 的成功主干，本章逐行拆它的错误分支（`IPCThreadState.cpp:916-990`）——这是全书出现频率最高的一段"排障词典"的原文出处。

14.1 错误的三种诞生地

诞生地一：发送端组包失败。 `writeTransactionData`（IPCThreadState.cpp:1104-1135）先对发送 Parcel 做 `errorCheck()`：数据非法（如 mError 已带病）时事务根本不出进程——此时 `transact` 直接把错误塞进 `mLastError` 返回（`:726-729`），驱动毫不知情。注意 `statusBuffer` 参数：BC_REPLY 组包失败时走 `TF_STATUS_CODE` 分支（`:1126-1131`），把错误码本身当作"数据"塞进事务——服务端至少能收到一个明确的"你的回包发不出去"。

诞生地二：驱动拒绝。 驱动在解析、寻址、分配缓冲任一步失败，都会以三个 BR 错误码把发送线程叫醒：`BR_DEAD_REPLY`（对端进程已死/引用悬空）、`BR_FAILED_REPLY`（事务被拒——async 配额耗尽、缓冲不足、fd 翻译失败等）、`BR_FROZEN_REPLY`（目标进程被冻结）。libbinder 侧的翻译只有四行（`:925-935`）：前两者映射为 `DEAD_OBJECT` 与 `FAILED_TRANSACTION`，后者伪装成 FAILED_TRANSACTION——**驱动侧并不知道"为什么失败"的细节，用户态更不知道**，这是 Binder 排障最大的信息缺口（排障手册场景二展开）。

诞生地三：服务端执行异常。 服务端业务代码抛出的异常走的是**数据通道**而非错误命令：Java 层 `reply.writeException()` 把异常编码成 reply Parcel 的特殊头部，原路送回，客户端 `readException()` 解码重抛——因为"业务语义的失败"对协议层是正常事务（第三章 3.1 的命令对里没有"异常"命令）。与它同层的还有 binder 级 `TF_STATUS_CODE`：`waitForResponse` 的 BR_REPLY 分支（:955-960）检查 flags，命中则把 4 字节数据直接当 `status_t` 读出——空错误不占对象槽，一套缓冲两种语义。

cpp frameworks/native/libs/binder/IPCThreadState.cpp:897-990（waitForResponse：错误分支逐行）

```
status_t IPCThreadState::waitForResponse(Parcel *reply, status_t *acquireResult)
{
uint32_t cmd;
int32_t err;
while (1) {
if ((err=talkWithDriver()) < NO_ERROR) break; // 讲解注释①：ioctl 本身失败（fd 失效等）
err = mIn.errorCheck();
if (err < NO_ERROR) break;
if (mIn.dataAvail() == 0) continue;
cmd = (uint32_t)mIn.readInt32();
switch (cmd) {
case BR_ONEWAY_SPAM_SUSPECT: // 讲解注释②：泛洪警告——只记账不改语义
ALOGE("Process seems to be sending too many oneway calls.");
CallStack::logStack("oneway spamming", ...); // 打印发送方调用栈（谁在刷 oneway）
[[fallthrough]]; // C++17 显式贯穿：警告后按 COMPLETE 处理
case BR_TRANSACTION_COMPLETE:
if (!reply && !acquireResult) goto finish; // 讲解注释③：oneway 的终点——
break; // 两参皆空说明这是单向调用，收件即返回
case BR_DEAD_REPLY:
err = DEAD_OBJECT; // 讲解注释④：对端死亡——句柄还在本进程
goto finish; //  （下次调用可能触发死亡通知，见第十六章）
case BR_FAILED_REPLY:
err = FAILED_TRANSACTION; // 讲解注释⑤：被驱动拒绝——原因最不透明
goto finish; //  （async 耗尽/缓冲不足/格式非法……）
case BR_FROZEN_REPLY:
err = FAILED_TRANSACTION; // 讲解注释⑥：目标被 cgroup 冻结——
goto finish; // 与 FAILED 合流，靠排障区分（10.4）
case BR_REPLY:
{
binder_transaction_data tr;
err = mIn.read(&tr, sizeof(tr));
if (reply) {
if ((tr.flags & TF_STATUS_CODE) == 0) { // 讲解注释⑦：正常回包——零拷贝包装
reply->ipcSetDataReference(..., freeBuffer);
} else { // 讲解注释⑧：状态码回包——4 字节即全部
err = *reinterpret_cast<const status_t*>(tr.data.ptr.buffer);
freeBuffer(nullptr, ...); // 立刻归还缓冲，不进 reply
}
} else {
freeBuffer(nullptr, ...); // 讲解注释⑨：调用方不想要回包（fakeReply）
continue; // 也不能泄漏缓冲——立刻归还
}
}
goto finish;
default:
```

```
err = executeCommand(cmd); // 讲解注释⑩：其余命令（引用计数/死亡通知…）
if (err != NO_ERROR) goto finish; // 在这里搭车处理——命令泵的复用
break;
}
}
finish:
if (err != NO_ERROR) {
if (reply) reply->setError(err); // 讲解注释⑪：错误写进 reply 对象本身
mLastError = err; // 线程级"最后一次错误"便于事后取证
}
return err;
}
```

设计洞察 · 逐段精读：BR_TRANSACTION_COMPLETE 与 sendReply 的成对关系

同步调用会收到**两次**"事务完成"类信号：先 BR_TRANSACTION_COMPLETE（驱动已收下我的 BC_TRANSACTION——此时 reply 参数非空，不退出循环），后 BR_REPLY（结果到了，退出）。而 oneway 只等第一次。服务端回包走 sendReply（:887）：writeTransactionData(BC_REPLY) 后调用 waitForResponse(nullptr, nullptr)——两参皆空，于是它同样在 BR_TRANSACTION_COMPLETE 处返回（③处条件命中）：**服务端确认"回包已被驱动收走"即继续服务下一个请求，不等于对端收到了。**COMPLETE 的语义是"我的半场结束了"，整场结果的见证者只有等待 BR_REPLY 的原线程。

14.2 从 status_t 到 Java 异常：最后一跳的翻译官

错误码穿过 libbinder 后，由 JNI 的 signalExceptionForError（android_util_Binder.cpp:828-900）翻译成 Java 异常。这张表里最有戏剧性的是 FAILED_TRANSACTION 的**双重人格**：

cpp frameworks/base/core/jni/android_util_Binder.cpp:874-900（节选：一错两掷）

```
case FAILED_TRANSACTION: {
ALOGE("!!! FAILED BINDER TRANSACTION !!! (parcel size = %d)", parcelSize); // 讲解注释①
const char* exceptionToThrow;
char msg[128];
if (canThrowRemoteException && parcelSize > 200*1024) {
// 讲解注释②：大包失败 → TransactionTooLargeException（受检异常）
exceptionToThrow = "android/os/TransactionTooLargeException";
snprintf(msg, sizeof(msg)-1, "data parcel size %d bytes", parcelSize);
} else {
// 讲解注释③：小包失败 → 大概率对端死了（也可能是缓冲耗尽）
exceptionToThrow = (canThrowRemoteException)
? "android/os/DeadObjectException" : "java/lang/RuntimeException";
snprintf(msg, sizeof(msg) - 1,
"Transaction failed on small parcel; remote process probably died, but "
"this could also be caused by running out of binder buffer space");
}
jniThrowException(env, exceptionToThrow, msg);
```

讲解注释①就是你搜遍 logcat 的那行 !!! FAILED BINDER TRANSACTION !!!——括号里的 parcel size 是 200KB 启发式的全部依据：**同一个错误码，size 大于阈值抛 TransactionTooLargeException（"你的包太大了"），小于阈值抛 DeadObjectException（"多**

半是对方死了"）。源码注释（:879-884）坦承这是无奈的启发式——驱动没把拒绝原因细分给用户态，于是 libbinder 用"包大小"猜原因。理解这一点，你就明白为什么 catch TransactionTooLargeException 时对方可能活得好好的（其实是缓冲紧张），以及为什么 DeadObjectException 有时其实是"async 空间满了"（排障手册场景二）。其余常见映射：`PERMISSION_DENIED`→SecurityException、`UNKNOWN_TRANSACTION`→RuntimeException、`NO_MEMORY`→OutOfMemoryError——在 `BinderProxy.transactNative`（BinderProxy.java:526 起的 JNI 调用链）尾端统一落网。

14.3 oneway 的死信语义：失败也可能无感

同步调用失败必然在调用线程炸出异常；oneway 失败却可能**静默**——这正是第十章 10.3 三个误区的第四块拼图。梳理 oneway 的四种结局：

- **顺利入队**：驱动写进目标 async_todo，发送线程收到 `BR_TRANSACTION_COMPLETE`（②处）即返回。目标什么时候执行、执行结果如何，协议层一概不知。
- **async 配额耗尽**：驱动在分配缓冲阶段拒绝，回 `BR_FAILED_REPLY`——**发送线程仍会拿到 FAILED_TRANSACTION 异常**（⑤处）。"oneway 失败无感知"只对"入队后"成立，入队前失败是能感知的。
- **对端死在入队后**：事务还躺在死进程的 todo 里，驱动回收缓冲后回 `BR_DEAD_REPLY`（④处）——同样炸 DEAD_OBJECT。真正的静默区间只有"已投递、已执行"的窄缝。
- **泛洪被点名**：`BR_ONEWAY_SPAM_SUSPECT`（②处）在 13 中默认开启（`open_driver` 里 `BINDER_ENABLE_ONEWAY_SPAM_DETECTION`，ProcessState.cpp:476），异常高频者被打印调用栈——但它**只警告不拦截**（fallthrough 继续），设计定位是"可观测、不改变语义"。

> **设计洞察 · 设计洞察：为什么错误也走数据通道**
>
> 三个诞生地里，服务端异常与 TF_STATUS_CODE 都借道"数据平面"而非新增 BR 命令。这不是偷懒：驱动为每笔事务分配的缓冲是"一次拷贝"的稀缺资源，错误信息通常只有 4 字节，若为它单开命令类型，命令流与数据流就要双通道维护；复用数据平面后，"空错误"只占一个 flag 位（TF_STATUS_CODE），重错误照常携带 offsets 表（跨进程异常还能顺路传回嵌套的 flat_binder_object，如 Parcelable 的 ParcelableException）。**协议的克制再次获胜：失败的事务仍是事务**，排障时 dump 出的错误事务与正常事务同构。

源码锚点 · 第十四章锚点

IPCThreadState.cpp:1104 writeTransactionData（TF_STATUS_CODE : 1126）/ :887 sendReply
:916 BR_ONEWAY_SPAM_SUSPECT / :921 BR_TRANSACTION_COMPLETE / :925/929/933 DEAD/FAILED/FROZEN
:955 BR_REPLY 的 TF_STATUS_CODE 分支 / :772 oneway 终点
android_util_Binder.cpp:875 !!! FAILED BINDER TRANSACTION / :885 200KB 启发式
UAPI binder.h:319 TF_STATUS_CODE / :488 BR_ONEWAY_SPAM_SUSPECT（:334 TF_ONE_WAY）

第十五章 三域、binderfs 与稳定性等级

设计洞察 · 学习目标

1. 说清三设备与三个servicemanager的对应；2. 解释binderfs动态设备与特性探测；3. 逐行读懂BINDER_SET_CONTEXT_MGR_EXT；4. 用位运算推演稳定性check

第一章 1.3 用一段血统史带过了 binderfs，第十章 10.5 列过稳定性等级——本章把这两条线合成一张**分区治理**的图：Android 为什么要开三个 binder 设备？binderfs 如何把它们从"写死"变成"动态"？接口稳定性等级的位编码藏着什么巧思？这些问题共享同一个答案的骨架：**Binder 的对象能力模型要跨安全边界（system/vendor），于是协议层长出了分区身份的三处肉身**——默认驱动选择（编译期）、事务头徽章（每笔事务）、稳定性尾注（每个对象）。链路位置：本章站在第七章（servicemanager）与第十三章（kHeader/尾注）之间，解释"名录为什么有三个、句柄 0 为什么互不相通"。

15.1 三域：一个驱动，三份名录

三域是治理需求倒逼的协议设计。system_server 与 App 共用 `/dev/binder`；HAL 进程与 client 共用 `/dev/hwbinder`；vendor 进程间的内网走 `/dev/vndbinder`。**同一份 binder 驱动代码，按打开的设备节点实例化出三个互不相通的上下文**——每个节点一棵独立的节点/引用树、一个独立的上下文管理者。跨域调用被从寻址层面切断：你在 vndbinder 域拿到的句柄，到 binder 域毫无意义，因为驱动的 refs 表按节点隔离。配合 SELinux（每个域的 allow 规则独立）与独立的名录进程，形成三层同心圆：设备隔离（硬）、SELinux 隔离（软硬之间）、服务名隔离（软）。

用户态的选择是编译期的（`ProcessState.cpp:52-54`）：`__ANDROID_VNDK__` 编译的 vendor 代码默认打开 `/dev/vndbinder`，其余默认 `/dev/binder`。三个域各有名录：system 的 `servicemanager`（servicemanager.rc，critical 属性、死而复生还负责重启 apexd 等伙伴）；vendor 的 `vndservicemanager`（vndservicemanager.rc 首行就是 `service vndservicemanager /vendor/bin/vndservicemanager /dev/vndbinder`）；hwbinder 的 servicemanager 同理（

Android 11 起由 servicemanager 二进制以参数区分，7.1 的 `main.cpp:113` 第一行 `argv[1]` 就是驱动路径）。注意 `ProcessState::init` 的兜底（`:112-116`）：驱动节点不可访问时告警并退回 `/dev/binder`——vendor 进程在非标准环境（如主机模拟）靠它活下来。

15.2 binderfs：设备从写死到动态

旧世界的问题：binder 设备是 misc 设备，主次设备号写死在内核注册表，容器/多用户要新增域就得改内核。2019 年合入的 binderfs 把设备节点变成可挂载文件系统里的**普通文件**——`mkdir /dev/binderfs && mount -t binder binder /dev/binderfs` 后，向 `binder-control` 写一个 `binderfs_device{name}` 结构（ioctl `BINDER_CTL_ADD`，UAPI `binderfs.h`，`BINDERFS_MAX_NAME=255`）即可实例化任意名字的新设备。Android 13 的 init.rc 已经全面迁移（`system/core/rootdir/init.rc:270-282`）：

text system/core/rootdir/init.rc:268-282（节选：binderfs 上场）

```
# Mount binderfs
mkdir /dev/binderfs // 讲解注释①：先造挂载点
mount binder binder /dev/binderfs stats=global
// 讲解注释②：挂 binderfs——stats=global 表示
// 全局 binder 统计走这一个挂载实例
chmod 0755 /dev/binderfs
symlink /dev/binderfs/binder /dev/binder // 讲解注释③：三个软链接保持老路径兼容——
symlink /dev/binderfs/hwbinder /dev/hwbinder // 一切还在写死路径的代码照常工作
symlink /dev/binderfs/vndbinder /dev/vndbinder
chmod 0666 /dev/binderfs/hwbinder // 讲解注释④：0666——所有人可读写设备，
chmod 0666 /dev/binderfs/binder // 因为访问控制已交给 SELinux（每域规则），
chmod 0666 /dev/binderfs/vndbinder // DAC 层不再重复设防
// 讲解注释⑤：/dev/binderfs 还暴露 features/ 目录——用户态探测驱动能力的地方，
// 例如 oneway_spam_detection 文件存在与否决定 ProcessState 是否抱怨 ioctl 失败
```

这个 features 目录是一条漂亮的**能力协商协议**（`ProcessState.cpp:413-418`）：libbinder 在 ioctl 新命令前先检查 `/dev/binderfs/features/oneway_spam_detection` 是否存在，不存在（老内核）就静默跳过，只在"本应支持却失败"时告警——驱动协议升级而用户态兼容性不破。binderfs 对厂商还有一个隐性好处：`vndbinder` 与 `binder` 在文件系统层面同源，多容器（Android 容器/车载多座舱）可按需 `BINDER_CTL_ADD` 出 `mycontainer-binder` 这样的私有域，每个域再配上自己的 context manager（15.3）。

设备节点的打开过程正是上述一切的总装线。第四章 4 的 ProcessState 构造器一行 `open_driver(driver)` 带过了，现在逐行读完它——三次 ioctl 恰好对应三道兼容性哲学：

cpp frameworks/native/libs/binder/ProcessState.cpp:452-486（open_driver 全文走读）

```
static base::Result<int> open_driver(const char* driver) {
int fd = open(driver, O_RDWR | O_CLOEXEC);
// 讲解注释①：O_CLOEXEC——binder fd 不许跨 exec 泄漏给子进程；
// fork 后使用 ProcessState 会被 verifyNotForked(:99) 直接 fatal
if (fd < 0) {
return base::ErrnoError() << "Opening '" << driver << "' failed";
}
int vers = 0;
```

```
status_t result = ioctl(fd, BINDER_VERSION, &vers);
// 讲解注释②：第一道 ioctl=协议版本握手（UAPI :288 _IOWR('b',9)）
// 用户态与驱动各自报号，谈不拢立刻分手——防止"半 compatible"通信
if (result == -1) {
close(fd);
return base::ErrnoError() << "Binder ioctl to obtain version failed";
}
if (result != 0 || vers != BINDER_CURRENT_PROTOCOL_VERSION) {
close(fd);
return base::Error() << "Binder driver protocol(" << vers
<< ") does not match user space protocol("
<< BINDER_CURRENT_PROTOCOL_VERSION
<< ")! ioctl() return value: " << result;
// 讲解注释③：版本错配是排障高频——内核跟错分支/刷错 GKI 时
// logcat 第一句就是这条；也解释了为什么 protocol version 必须
// 是打开后的第一句话（后面一切结构布局都依赖它）
}
size_t maxThreads = DEFAULT_MAX_BINDER_THREADS;
result = ioctl(fd, BINDER_SET_MAX_THREADS, &maxThreads);
// 讲解注释④：第二道 ioctl=声明线程预算（15，第四章 :470）
// 注意只是"报预算"——驱动此后按并发压力发 BR_SPAWN_LOOPER 扩池，
// 用户态从不主动拉线程（供给制，第十章）
if (result == -1) {
ALOGE("Binder ioctl to set max threads failed: %s", strerror(errno));
}
uint32_t enable = DEFAULT_ENABLE_ONEWAY_SPAM_DETECTION;
result = ioctl(fd, BINDER_ENABLE_ONEWAY_SPAM_DETECTION, &enable);
// 讲解注释⑤：第三道 ioctl=开启泛洪检测（13 新协议 :298 _IOW('b',16)）
// 失败时先查 /dev/binderfs/features/oneway_spam_detection——
// 特性根本不存在（老内核）则静默，存在却失败才告警（能力协商）
if (result == -1) {
ALOGE_IF(ProcessState::isDriverFeatureEnabled(
ProcessState::DriverFeature::ONEWAY_SPAM_DETECTION),
"Binder ioctl to enable oneway spam detection failed: %s", strerror(errno));
}
return fd;
// 讲解注释⑥：注意这里没有 mmap——映射发生在 ProcessState 构造器里
// （第一章 :503），open 与 mmap 分离让错误可归因：开不了=驱动问题，
// 映射不了=地址空间问题
}
```

三次 ioctl 的顺序就是一条兼容性叙事：**版本（必须一致，否则拒绝工作）→ 线程预算（失败仅告警，缺它进程退化为单线程服务）→ 泛洪检测（失败仅告警，缺它少一层观测）**。新功能的优先级与回退策略在打开的一瞬间全部定完，之后 runtime 不再做任何特性判断——这是"启动时协商、运行时零分支"的工程范式。

15.3 成为上下文管理者：BINDER_SET_CONTEXT_MGR 的现代形态

第七章 7 的 `main.cpp:134` 一行 `becomeContextManager()`，内部是这个样子：

cpp frameworks/native/libs/binder/ProcessState.cpp:191-210（成为名录的 ioctl 全文）

```
bool ProcessState::becomeContextManager()
{
```

```
AutoMutex _l(mLock);
flat_binder_object obj { // 讲解注释①：EXT 版本要带一个对象——
.flags = FLAT_BINDER_FLAG_TXN_SECURITY_CTX, // 申请驱动在发来的每笔事务上
}; // 附带发送方 SELinux 上下文字符串
int result = ioctl(mDriverFD, BINDER_SET_CONTEXT_MGR_EXT, &obj);
// 讲解注释②：UAPI binder.h:296——_IOW('b', 13, struct flat_binder_object)
// 上下文管理者的身份+行为偏好打包成一个 flat_binder_object 传下去
// fallback to original method
if (result != 0) {
android_errorWriteLog(0x534e4554, "121035042"); // 讲解注释③：0x534e4554='SENT',
// Android 安全 bug 编号入库的固定仪式
int unused = 0;
result = ioctl(mDriverFD, BINDER_SET_CONTEXT_MGR, &unused); // 讲解注释④：老内核
} // 退回老 ioctl ('b',7,__s32)
if (result == -1) {
ALOGE("Binder ioctl to become context manager failed: %s\n", strerror(errno));
}
return result == 0;
}
```

三个值得咀嚼的细节。其一，**EXT 与老接口并存**是 Android 内核 ABI 演进的范本：新能力（事务附带安全上下文，支撑 5.2 的 `mCallingSid`）用新 ioctl，老内核静默回退，功能降级但不崩溃。其二，同一进程**只能有一个**上下文管理者：后来者 ioctl 返回 EBUSY——"抢名录"在驱动层就不可能成功。其三，`BINDER_SET_CONTEXT_MGR` 的管理范围是**当前打开的设备节点所代表的 context**（EXT 版本的参数 flat_binder_object 与老版本 `binder.h:291-296` 的 `_IOW('b',7/13,…)` 编号可从 UAPI 直接核对）：打开哪个设备节点，就成为哪个域的名录——三个 servicemanager 就是三次这样的 ioctl。每个域的句柄 0 因此天然不同，跨域的"句柄 0"互为陌生人，这是 15.1 设备隔离在协议层的最后封印。

15.4 稳定性等级：一位一世界的位编码

第十章 10.5 说了等级的用途，本章拆它的编码——因为 `Stability.h:133-138` 的定义值得背诵：

text frameworks/native/libs/binder/include/binder/Stability.h:133-138

```
enum Level : int16_t {
UNDECLARED = 0,
VENDOR = 0b000011, // 只承诺 vendor 分区内可用
SYSTEM = 0b001100, // 只承诺 system 分区内可用
VINTF = 0b111111, // 承诺全设备（vendor interface）可用
};
// 讲解注释①：check 的实现只有一行位运算（Stability.cpp:155-157）
// bool stable = (provided & required) == required;
// 讲解注释②：推演——required=VENDOR(0b000011) 时，provided=VINTF(0b111111)
// 按位与=0b000011==required ✓（VINTF 满足 vendor 需求）
// provided=SYSTEM(0b001100)&0b000011=0b000000 ✗（system 接口喂不给 vendor）
// 讲解注释③：位图=承诺域的集合！VENDOR 是低两位、SYSTEM 是中两位，
// VINTF 全一=两域都承诺。"更稳"的字面意思就是"更多的 1"
```

等级从哪里来？`getLocalLevel()`（Stability.cpp:76-84）：`__ANDROID_VNDK__` 编译产出 VENDOR、否则 SYSTEM——又是编译期分区身份。首次跨进程传递时 `finishFlattenBinder`（13.3 ⑥）调用 `tryMarkCompilationUnit` 把等级"定型"进 BBinder 的 mStability 字段，此后随 4 字节尾注环游世界；接收方 `setRepr` 校验**已定型的等级不可更改**（`:117-123`，只能显式允许的降级）。最后一

道闸在 `BpBinder::transact`（BpBinder.cpp:286-297）：发起用户事务前，用本进程等级作 required 做 check，不过则打日志 `Cannot do a user transaction on a %s binder (%s) in a %s context.` 并返回 BAD_TYPE——**vendor 进程调用 system 稳定性的接口，在客户端本地就被拦下，事务根本不上路**。加上 FLAG_PRIVATE_VENDOR（:281，要求 required 提到 VENDOR）这一防"vendor 对象泄漏给 system"的补充标记，稳定性体系闭合。

设计洞察 · 设计洞察：分区身份的三处肉身

同一份"我是谁"的编译期事实（__ANDROID_VNDK__），在运行时以三种形态反复出现：ProcessState 的默认驱动选择（15.1，决定进哪个域）、Parcel 的 kHeader 徽章（13.2，随每笔事务声明出身）、Stability 的 localLevel（15.4，决定能向谁承诺）。这是"分布式系统里的身份"一课：**身份必须在每一个边界上重新出示**，任何一处只靠前一处的传递都会被伪造或遗漏——Binder 把同一事实冗余三次，换来三类攻击面（域混用、来源伪装、等级欺骗）各自独立闭合。

源码锚点 · 第十五章锚点

ProcessState.cpp:52-54 kDefaultDriver 三域选择 / :112-116 /dev/binder 兜底
:191-210 becomeContextManager（EXT :199 / 回退 :206）/ :413-418 binderfs features 探测
system/core/rootdir/init.rc:270 mount binderfs / :276-278 三 symlink
UAPI binderfs.h BINDER_CTL_ADD+binderfs_device / binder.h:291-296 CONTEXT_MGR(_EXT)
Stability.h:133 Level 位图 / Stability.cpp:155 check 位运算 / BpBinder.cpp:286-297 稳定闸门

第十六章 死亡通知全链路与重连模式库

设计洞察 · 学习目标

1. 复述七跳死亡链的每一站；2. 解释incWeak与BC_DEAD_BINDER_DONE的记账闭环；3. 说出四种重连模式各自的适用场景；4. 识别三种死亡通知反模式

第十章 10.1 讲了死亡通知"是什么"（BC_REQUEST_DEATH_NOTIFICATION → BR_DEAD_BINDER），本章把这条线拉全长：一根从 Java 的 `DeathRecipient` 回调出发、穿过 JNI、潜入 libbinder、抵达驱动登记表、再原路折返的完整旅程——每一站的类名、行号与所有权转移都核实过。链路位置：它是第三章命令对表里最复杂的一对（涉及 BC/BR 之外的 DONE 确认命令），是第七章自动除名（`ServiceManager.cpp` 的 linkToDeath）的引擎，也是 8.1 图里 DeathRecipientList 那条缝合线的展开。掌握了它，第九章"runtime restart 后句柄失效"才真正闭环。

16.1 七跳全链：从 linkToDeath 到 binderDied

以 Java 层一次 `proxy.linkToDeath(recipient, 0)` 为例（`BinderProxy.java:619` 的 native 方法）：

text 死亡通知七跳（收报方视角从上往下，出殡方从第三跳折返）

```
①Java BinderProxy.linkToDeath(DeathRecipient, int) BinderProxy.java:619
→ JNI：android_os_BinderProxy_linkToDeath android_util_Binder.cpp:1446
②JNI new JavaDeathRecipient(env, recipient, list) :559（全局引用持有 Java 回调）
DeathRecipientList（BinderProxyNativeData.mOrgue） :739 每代理一本讣告簿
③Native BpBinder::linkToDeath(jdr, cookie, flags) BpBinder.cpp:326
mObituaries 登记 + requestDeathNotification(handle) :350 IPCThreadState.cpp:850
→ mOut: BC_REQUEST_DEATH_NOTIFICATION + flushCommands :353
④驱动 在该 handle 对应的 binder_ref 上登记 cookie=BpBinder 指针
—— UAPI 反推：驱动在 node 销毁时按 ref 逐个回推 BR_DEAD_BINDER
⑤泵 executeCommand(BR_DEAD_BINDER) IPCThreadState.cpp:1352
proxy->sendObituary() + BC_DEAD_BINDER_DONE :1354-1356
⑥Native BpBinder::sendObituary → reportOneDeath(promote) BpBinder.cpp:400/435
→ JavaDeathRecipient::binderDied(who) android_util_Binder.cpp:572
⑦Java BinderProxy.sendDeathNotice(recipient, proxy) BinderProxy.java:699
→ recipient.binderDied(binderProxy)——你的代码在这里
// 讲解注释①：cookie 在③⑥之间是"回程邮票"——驱动只存指针不懂语义，原样带回
// 讲解注释②：⑤的 BC_DEAD_BINDER_DONE 是应用向驱动的签收：驱动据此回收登记表项
// 讲解注释③：第④跳处于驱动侧，本树无 binder.c，按 UAPI 命令对与 libbinder 行为反推
```

下面逐行精读第③跳与第⑤⑥跳的 libbinder 原文——所有权的巧思集中在这里：

cpp frameworks/native/libs/binder/BpBinder.cpp:326-360 + IPCThreadState.cpp:1352-1365（拼合走读）

```
status_t BpBinder::linkToDeath(const sp<DeathRecipient>& recipient, void* cookie, uint32_t
flags)
{
Obituary ob; // 讲解注释①：Obituary=讣告（recipient+cookie+flags）
ob.recipient = recipient;
ob.cookie = cookie;
{
AutoMutex _l(mLock);
if (!mObitsSent) { // 讲解注释②：讣告已发出过？死后订阅不可能——
if (!mObituaries) { // 直接 DEAD_OBJECT，别指望"补挂号"
mObituaries = new Vector<Obituary>;
ALOGV("Requesting death notification: %p handle %d\n", this, binderHandle());
getWeakRefs()->incWeak(this); // 讲解注释③：死亡登记替 BpBinder 持一个弱计数
// ——驱动侧登记在册期间代理不许清零销毁
IPCThreadState* self = IPCThreadState::self();
self->requestDeathNotification(binderHandle(), this); // 讲解注释④：三连发：
self->flushCommands(); // BC_REQUEST_DEATH_NOTIFICATION(handle,
} // BpBinder指针）立刻冲进驱动（:850-857）
ssize_t res = mObituaries->add(ob); // 同一 BpBinder 可挂多个讣告——向量累积
return res >= (ssize_t)NO_ERROR ? (status_t)NO_ERROR : res;
}
}
return DEAD_OBJECT; // 讲解注释⑤：活着才可订阅；已死请走异常路径
}
case BR_DEAD_BINDER: // 讲解注释⑥：executeCommand 收到讣告
{
```

```
BpBinder *proxy = (BpBinder*)mIn.readPointer(); // cookie 回来了——就是当年登记的 this
proxy->sendObituary(); // 通知所有登记者（⑥跳入口）
mOut.writeInt32(BC_DEAD_BINDER_DONE); // 签收回执：驱动可以回收这条登记了
mOut.writePointer((uintptr_t)proxy);
} break;
case BR_CLEAR_DEATH_NOTIFICATION_DONE: // 讲解注释⑦：unlink 或讣告发出后的
{ // 清理确认——驱动让出那个弱计数
BpBinder *proxy = (BpBinder*)mIn.readPointer();
proxy->getWeakRefs()->decWeak(proxy); // 与③的 incWeak 严格配对
} break;
```

设计洞察 · 逐段精读：三重生命周期的咬合

这条链最精妙的是**谁替谁计数**：Java 的 DeathRecipient 被 JNI 全局引用强持有（①跳，:560）；JavaDeathRecipient 被 DeathRecipientList 强持有、List 又被 BinderProxyNativeData.mOrgue 强持有（与 BinderProxy 同生共死）；BpBinder 则被③的 incWeak 护住——**只要驱动里还有一条死亡登记，代理就保持"弱可达"**，直到 BR_DEAD_BINDER 派出讣告并收到 BC_DEAD_BINDER_DONE、或 unlink 触发 CLEAR_DEATH_NOTIFICATION_DONE，⑦处才 decWeak 放手。任何一跳丢失，要么泄漏（没 unlink、没 DONE），要么悬垂（提前 decWeak）。读到这里再回头看 8.1 的"双向生命周期由 DeathRecipientList 缝合"，每一个箭头都有了对象与行号。

`sendObituary`（BpBinder.cpp:400-440）内部还有两处防线的细节：`mObitsSent` 原子翻转保证讣告**只发一次**（多次 BR_DEAD_BINDER/竞态 linkToDeath 不会重复回调）；`reportOneDeath` 里 `obit.recipient.promote()`（:437）先提升弱引用——**回调发出前允许接收方先死**（提升失败静默跳过），Java 侧对应 `JavaDeathRecipient::binderDied` 收尾时把强全局引用降级为弱引用（android_util_Binder.cpp:593-596），让"已死服务的通知簿"不再阻止 GC。异常处理同样有交代：`BinderProxy.sendDeathNotice`（:699-708）把回调里抛出的 RuntimeException 捕获并只打日志——**死亡通知的回调坏了不能拖垮 binder 线程**。

16.2 重连模式库：服务死了之后怎么办

拿到讣告只是开始，工业级代码要回答"下一步"。框架内验证过的四个模板，按"谁主动"排序：

模板一：waitForService 挂起等待（服务端主导，推荐）——`ServiceManagerShim::waitForService`（IServiceManager.cpp:338-420）：先 checkService 一次（热路径零开销），未命中向名录 `registerForNotifications(name, waiter)` 注册 BnServiceCallback，然后**条件变量挂起**，服务的 onRegistration 回调把它叫醒。注意其源码注释：唤醒回调要由线程池线程送达，所以等待者必须 `wait_for` 带 1 秒超时兜底，醒来后还要重试 getService 处理 lazy 服务的竞态（服务死了→名录除名→init 重启→你又拉起，:405-415 的注释把这五步竞态写成了散文）。这是 Android 13 名录客户端的标准姿势：

cpp frameworks/native/libs/binder/IServiceManager.cpp:338-420（waitForService 走读）

```
sp<IBinder> ServiceManagerShim::waitForService(const String16& name16)
{
class Waiter : public android::os::BnServiceCallback {
Status onRegistration(const std::string& /*name*/,
const sp<IBinder>& binder) override {
```

```
// 讲解注释①：服务出现时名录回调这里——运行在调用方的 binder 线程池里
std::unique_lock<std::mutex> lock(mMutex);
mBinder = binder; // 讲解注释②：先落结果再唤醒（锁内写、锁外唤醒）
lock.unlock();
IPCThreadState::self()->flushCommands();
// 讲解注释③：立刻冲一次 ioctl——引用计数的 BC_ACQUIRE 不能滞留 mOut，
// 否则"我持有引用"这件事对端暂时不知道（第四章 flushCommands 复用）
mCv.notify_one();
return Status::ok();
}
public:
sp<IBinder> mBinder; // 讲解注释④：结果位——非空即"服务到了"
std::mutex mMutex;
std::condition_variable mCv;
};
const std::string name = String8(name16).c_str();
sp<IBinder> out;
if (Status status = realGetService(name, &out); !status.isOk()) { ... }
if (out != nullptr) return out; // 讲解注释⑤：热路径——服务已在，零注册开销
sp<Waiter> waiter = sp<Waiter>::make();
if (Status status = mTheRealServiceManager->registerForNotifications(name, waiter);
!status.isOk()) { ... } // 讲解注释⑥：向名录订阅"出现"事件（模板四）
Defer unregister ([&] {
mTheRealServiceManager->unregisterForNotifications(name, waiter);
}); // 讲解注释⑦：RAII——任何退出路径必注销订阅
while(true) {
{
std::unique_lock<std::mutex> lock(waiter->mMutex);
waiter->mCv.wait_for(lock, 1s, [&] {
return waiter->mBinder != nullptr;
});
// 讲解注释⑧：必须带 1 秒超时——回调依赖线程池线程送达，
// 若本进程线程池未起/被占满，notify 永远不来，纯 wait 会挂死
if (waiter->mBinder != nullptr) return waiter->mBinder;
}
ALOGW("Waited one second for %s ...", name.c_str());
// 讲解注释⑨：lazy 服务竞态兜底——服务死亡瞬间 getService 可能
// 撞上"名录已除名、init 尚未重启"的窗口，定期重试 realGetService
// 才能把 init 的重新拉起触发出来（源码 :405-415 注释的五步散文）
if (Status status = realGetService(name, &out); !status.isOk()) { ... }
if (out != nullptr) return out;
}
}
```

模板二：getService 轮询（遗留兼容，勿新用）——ServiceManagerShim::getService（:264-301）：checkService 失败后按 sys.boot_completed 决定 100ms/1000ms 步长轮询，5 秒上限。它至今存在的原因写在 :260-263 的注释里——改成 waitForService 会改变调用线程的阻塞结构、可能破坏预编译 binary 的时序假设。**新代码禁止模仿**：无通知、占线程、上限拍脑袋。

模板三：linkToDeath + 重建（客户端主导，有状态连接）——对"已经拿到句柄"的长连接，死亡通知是唯一正确的失效信号。框架最著名的用户是 AMS 的 AppDeathRecipient（10.1，ProcessList.java:5056）：进程死亡→AMS 清理 ProcessRecord→客户端重新 bindService。自研服务照抄三件事：收到 binderDied **先失效缓存再重连**（旧句柄已不可用）；重连用模板一（不自旋）

；unlinkToDeath 与 linkToDeath 严格成对（16.1 的计数闭环，否则泄漏并被 warnIfStillLive 点名，android_util_Binder.cpp:626-655）。

模板四：registerForNotifications（订阅"出现"）——ServiceManager.registerForNotifications（ServiceManager.java:308 的 native 对端 :338 模板一）。与死亡通知对称：一个通知"没了"，一个通知"有了"。适合缓存同步器——服务上线即刷新、下线即标记，两端事件合起来才是完整的生命周期流。

text 四模式速查（何时用哪个）

```
┌──────────────┬──────────┬──────────────┬──────────────────────────────┐
│ 模式 │ 触发时机 │ 典型场景 │ 框架实锤 │
├──────────────┼──────────┼──────────────┼──────────────────────────────┤
│ waitForService │ 启动时等依赖│ 系统服务互调 │ IServiceManager.cpp:338 │
│ getService轮询 │ 遗留兼容 │ 不要新用 │ IServiceManager.cpp:264 │
│ linkToDeath │ 持有句柄后 │ 长连接+状态缓存 │ ProcessList.java:5056 │
│ regForNotif │ 服务出现时 │ 缓存同步器 │ ServiceManager.java:308 │
└──────────────┴──────────┴──────────────┴──────────────────────────────┘
// 反模式一：binderDied 里直接 getService 死循环——binder 线程被占死
// 反模式二：linkToDeath 后假定永不回调、不处理"注册即已死"（②处 DEAD_OBJECT）
// 反模式三：缓存失效了但句柄没换——旧 BinderProxy 复活调用只会再吃 DEAD_OBJECT
```

设计洞察 · 设计洞察：死亡通知是能力模型的终结包

Binder 的对象引用是能力凭证（1.4），而凭证必须有注销机制，否则"死了的服务"仍占用引用者心智与句柄表。死亡通知本质上由**三个协议命令**共同完成：REQUEST（订阅）、DEAD（讣告）、DONE（签收）——比调用协议多一层"应用确认"，因为驱动的登记表也是内存（第三章五结构之外的第五维度账本），必须等应用明确"我处理完了"才能回收。第十六章之后再看十二章 12.1 的 debugfs：cat /d/binder/proc/<pid> 里 refs 项的 node/proc 与死亡登记一一对应——排障时"句柄为什么还活着/为什么提前死了"的答案就在那里。

源码锚点 · 第十六章锚点

BinderProxy.java:619 linkToDeath / :699 sendDeathNotice
android_util_Binder.cpp:555-665 JavaDeathRecipient / :572 binderDied / :1446 linkToDeath JNI
BpBinder.cpp:326 linkToDeath / :400 sendObituary / :435 reportOneDeath（promote : 437）
IPCThreadState.cpp:850 requestDeathNotification / :1352 BR_DEAD_BINDER+BC_DEAD_BINDER_DONE
:1360 BR_CLEAR_DEATH_NOTIFICATION_DONE（incWeak/decWeak 闭环）
IServiceManager.cpp:338 waitForService / :264 getService 轮询 / ServiceManager.java:308 registerForNotifications

教科书习题集（分级）

题号 1-14 为原题，15-20 为续编新题（按级归入各级末尾，配合第十三章至第十六章的新内容）；全部 20 题的参考答案要点见本节末尾。

第一级：简答题

1. Binder 的"一次拷贝"是怎么实现的？mmap 的是哪个进程的地址空间？

2. BC_TRANSACTION 和 BR_TRANSACTION 各是谁发给谁的？

3. BinderProxy 和 BpBinder 各在哪层？为什么需要两层？

4. oneway 的三个常见误区是什么？

5. servicemanager 为什么零线程池、仅 oneway？

6. Binder 的最大线程数是多少？system_server 是多少？为什么不同？

7. 事务即域（transaction-scoped）是什么意思？违反它会怎样？

8. linkToDeath 和 BR_DEAD_BINDER 的关系是什么？

15. Parcel 的"双平面"指什么？为什么驱动能"跳着扫"对象而不解析全部数据？

16. 同一个 FAILED_TRANSACTION 错误码，为什么 Java 侧可能是两种异常？判据是什么？

第二级：分析题

9. **追踪时序**：画一次 oneway 调用的时序图，标出与同步调用的分岔点。

10. **身份链**：AMS 在处理一个 App 的 startActivity 时，Binder.getCallingUid() 经历了怎样的变化？什么时候该 clearCallingIdentity？

11. **数据结构推演**：如果 App A 和 App B 同时持有 "activity" 服务的 binder_ref，驱动侧有几个 binder_node？几个 binder_ref？画出关系图。

12. **失败场景**：system_server 崩溃后，正在等 BR_REPLY 的 App 会收到什么？已经注册的服务会发生什么？

17. **追踪错误流**：画出一条 oneway 事务因 async 配额耗尽被拒的完整命令流（BC/BR 级别）。发送方最终收到什么异常？为什么"oneway 失败也可能感知"？

18. **计数闭环**：BpBinder::linkToDeath 里 incWeak 与 BR_CLEAR_DEATH_NOTIFICATION_DONE 的 decWeak 如何配对？应用忘记 unlinkToDeath 会发生什么？对同一个 recipient linkToDeath 两次呢？

第三级：设计题

13. **设计一个新 IPC**：假设 Binder 不存在，你会怎么设计 Android 的 IPC？从安全（身份）、性能（拷贝）、能力传递三个维度给出方案，并对比你的方案与 Binder 的优劣。

14. **调试实战**：用户报告"App 调用系统服务偶尔超时"，给你一台 userdebug 设备，设计一套完整定位方案（从 dumpsys 到 perfetto 到代码审查）。

19. **设计一个跨域服务**：某 vendor HAL 进程需要向 App 暴露一个接口，又要被 system_server 调用。用三域、稳定性等级、binderfs 与 VINTF 知识设计：接口标什么等级？注册在哪个域？App 的调用会在哪几道闸被检查？

20. **设计自动重连**：给第十六章模板一的 waitForService 增加"拿到句柄后死亡自动重连"能力。写出伪代码，并逐条说明它与三种反模式（死循环占 binder 线程/不失效缓存/不处理注册即死）的区别。

参考答案要点

1. 接收方 mmap 自己的地址空间（1MB-2页只读映射），驱动 copy_from_user 直写接收方的映射区。

2. 都是驱动与应用之间：BC_TRANSACTION 是应用→驱动（发送侧发起），BR_TRANSACTION 是驱动→应用（接收侧收到调用）。同一笔事务在发送方视角是 BC，在接收方视角是 BR。

3. BpBinder 在 native 层（句柄包装）、BinderProxy 在 Java 层（BpBinder 的 Java 马甲）。两层因语言边界而生：GC/反射世界需要 jobject 形态、每进程复用一个代理（8.1 的 getInstance），native 引用计数世界需要 sp 形态。

4. ①COMPLETE≠对端已执行；②同目标保序、跨目标乱序；③async 独立配额，塞满 FAILED_REPLY（10.3/14.3）。

5. 名录必须极简极稳：零线程池避免线程膨胀被泛洪牵连；FATAL_IF_NOT_ONEWAY 从机制上杜绝被慢客户端同步卡死（5.1/第七章）。

6. 普通进程 15+1=16；system_server 31+1=32（SystemServer.java:435）——框架进程并发需求高。

7. 事务即域=Parcel 数据由 binder 缓冲区背书，事务返回即失效。跨事务保存等于使用已回收内存（缓冲会被驱动复用），数据静默损坏。

8. linkToDeath 在 ref 上订阅，BR_DEAD_BINDER 是 node 销毁时驱动按订阅推送的讣告——订阅与推送一对命令（第十六章七跳）。

9. 分岔点只有 TF_ONE_WAY 一个比特：同步走⑤后继续等 BR_REPLY（⑨⑩回程），oneway 从 :772 进入 waitForResponse(nullptr,nullptr)、在 :921 收到 BR_TRANSACTION_COMPLETE 即返回，⑥的入队走 async_todo。

10. 收到事务时 getCallingUid=App uid；AMS 代行权限前 clearCallingIdentity 恢复 system 身份。

11. 1 个 binder_node（AMS 只有一个服务端点）；2 个 binder_ref（每个客户端进程在自己的 refs 树里各有一条）。句柄 desc 在两进程可能不同值。

12. 等 reply 的 App 收到 BR_DEAD_REPLY（DEAD_OBJECT 异常）；servicemanager 的 linkToDeath 触发自动除名。

13. 评分要点：安全=内核采集身份（不可用户态伪造）；性能=一次拷贝（mmap 双映射）；能力=引用即权限（ref/node 模型）。三者互为前提才成立，逐条对比管道/shm 的缺口。

14. 评分要点：复现率统计→dumpsys binder_calls_stats 锁接口→/d/binder/transactions 看 pending 与嵌套→perfetto 拆唤醒/执行/回复三段→最后代码审查（嵌套同步调用、慢业务、锁序）。

15. 数据平面 mData（字节）+对象平面 mObjects（偏移数组）。驱动只看偏移表逐个翻译 24 字节 flat_binder_object，数据全程不透明——控制与数据分离（13.1）。

16. JNI 的 signalExceptionForError 用 200KB 启发式（android_util_Binder.cpp:885）：parcel size 大抛 TransactionTooLargeException，小抛 DeadObjectException（"多半对端死了，也可能是缓冲耗尽"）。因为驱动不细分拒绝原因（14.2）。

17. BC_TRANSACTION(TF_ONE_WAY)→驱动 async 分配失败 →BR_FAILED_REPLY→waitForResponse :929 映射 FAILED_TRANSACTION→signalExceptionForError 按 size 抛 TOE/DOE。oneway 在"入队前"失败同样同步炸异常，静默只发生在"入队后"（14.3 四种结局）。

18. 首次 linkToDeath 时 incWeak+BC_REQUEST_DEATH_NOTIFICATION；unlink 清空讣告簿时发 BC_CLEAR_DEATH_NOTIFICATION；驱动回 BR_CLEAR_DEATH_NOTIFICATION_DONE 时 decWeak（IPCThreadState.cpp:1360-1364）。忘记 unlink=弱计数悬挂、泄漏并被 warnIfStillLive 点名；两次 link 同一 recipient=登记两条讣告，binderDied 回调两次（BpBinder.cpp:326-360）。

19. 评分要点：接口等级=VINTF（0b111111，App=system 域可调、HAL=vendor 域可调，15.4 位运算）；注册在 /dev/hwbinder 或经 servicemanager 统一名录（域选择编译期定，15.1）；闸检查=BpBinder.transact 稳定闸（BpBinder.cpp:286）+SELinux 每事务采集+名录 canFind。

20. 伪代码骨架：linkToDeath(rec)→binderDied(){清缓存→句柄置空 →handle=waitForService(name)→重新 asInterface→重挂 linkToDeath}。区别一：重连用条件变量挂起而非轮询自旋（不占 binder 线程）；区别二：先失效缓存后重连（旧代理不再被调用）；区别三：注册前先 checkService 一次处理"注册即已死"（waitForService :370 的热路径与 1 秒兜底）。

术语表（按拼音排序）

术语	所在册	解释
AIDL	3	Android Interface Definition Language，自动生成 Stub/Proxy
async space	3	oneway 事务的独立配额（约映射区一半）
BBinder	3	Native 服务端基类（onTransact 入口）
BC_*	3	应用→驱动的命令前缀（BC_TRANSACTION 等）
Binder	3	Android 核心 IPC：一次拷贝+内核身份+能力传递
BinderProxy	3	Java 层的远端代理（BpBinder 的马甲）
BpBinder	3	Native 层的远端代理（持 32 位句柄）
BR_*	3	驱动→应用的命令前缀（BR_TRANSACTION 等）
binder_proc	3	驱动侧的进程对象（threads/nodes/refs 树）
binder_ref	3	驱动侧的客户端引用（desc=句柄）
binder_node	3	驱动侧的服务端点（ptr/cookie 指向 BBinder）
binder_thread	3	驱动侧的线程对象（transaction_stack 栈）
flat_binder_object	3	Parcel 内的对象槽线格式（本地指针/远端句柄二态）
FROZEN_REPLY	3	12+ 冻结进程的同步调用快速失败
handle 0	3	servicemanager 的固定句柄（自举）
IPCThreadState	3	每线程的命令收发中枢（mIn/mOut 缓冲）
oneway	3	单向事务：BR_TRANSACTION_COMPLETE 即返回
Parcel	3	跨进程字节容器（事务即域）
ProcessState	3	每进程的 Binder 资产（open/mmap/线程池）

术语	所在册	解释
RPC backend	3	13 实验分支：Binder 协议跑在任意传输上
servicemanager	3	Binder 服务名录（零线程池+仅 oneway）
Stub/Proxy	3	AIDL 生成的服务端存根/客户端代理
transaction_stack	3	线程上的同步调用链栈（支持跨进程递归）
事务即域	3	Parcel 生命周期随事务——跨事务保留=使用已回收内存

排障手册：Binder 问题定位

每个场景按 **症状 → 定位命令 → 根因 → 修复** 四段展开，日志样例均取自真实源码字符串（括注出处行号）。通用兵器谱先备齐：`logcat -b all`（main+system+events+crash 四缓冲一起看）、`dumpsys binder_calls_stats`（接口级耗时统计，部分版本需先 `--enable`）、`cat /sys/kernel/debug/binder/`（stats/failed_transaction_log/transactions/proc/<pid>，userdebug 设备可读）、`debuggerd -b <pid>`（全部线程栈）、`perfetto` 的 binder_track（事务级时序）。

场景一：!!! FAILED BINDER TRANSACTION !!!

- **症状**：logcat 炸出 `ALOGE("!!! FAILED BINDER TRANSACTION !!! (parcel size = %d)", parcelSize)`（android_util_Binder.cpp:875），Java 侧抛 TransactionTooLargeException 或 DeadObjectException，消息可能是 `data parcel size N bytes` 或 `Transaction failed on small parcel; remote process probably died, ...`（:885-897）。
- **定位命令**：先看日志里的 parcel size——大于 200×1024（:885）判定为"真大包"；再看 `logcat -b crash` 拿到调用栈确定是哪个事务；`cat /sys/kernel/debug/binder/failed_transaction_log` 看驱动侧最近失败记录；对端若存在，**dumpsys binder <对端pid>** 查其缓冲水位（transactions 里的 free space）。
- **根因**：FAILED_TRANSACTION 是驱动拒绝的合流错误码（14.1）。三种典型：单笔事务超过目标进程 mmap 区剩余空间；同步缓冲被并发事务占满（1MB-2 页是全进程共享预算，不是每调用一份）；oneway 走 async 半区配额（约一半缓冲），大位图 oneway 必翻车。小包失败则是"对端已死"或"缓冲紧张"的伪装（14.2 的双重人格）。
- **修复**：大包改用 fd 传递（writeBlob Ashley 分支或 SharedMemory + writeFileDescriptor）；批量小事务合并为一次调用；oneway 大数据降级为同步或分片；确系对端死忙则修对端并发。**不要**把 TransactionTooLargeException 一律当"数据太大"修——小包失败指向的是对端状态（14.2）。

场景二：DeadObjectException 但对端进程还活着

- **症状**：调用系统服务或自研服务抛 `android.os.DeadObjectException`，但 ps -A | grep <对端> 显示进程健在；或异常消息是 `Transaction failed on small parcel; remote process probably died` 的反向——小包却被拒。
- **定位命令**：`logcat -b all | grep -E "BR_FAILED|binder"` 找驱动侧拒绝上下文；`cat /sys/kernel/debug/binder/transactions` 看是否有进程 pending 在对端；`dumpsys activity processes | grep -E "nonPaced|binder"` 辅助；若怀疑服务重启过，`adb shell service list` 对比句柄时代（第九章 IActivityManagerSingleton 缓存失效叙事）。
- **根因**：BR_DEAD_REPLY 不只对"进程死亡"——驱动在目标 node 已销毁、句柄引用悬空时同样回它。常见三因：服务曾崩溃重启（句柄失效但进程重启了，同名新 node 与你的旧 ref 无关）；缓存代理未随死亡通知刷新（第九章）；oneway 事务在死掉的旧实例 todo 里被回收。
- **修复**：客户端实现死亡通知失效链（第十六章模板三：binderDied 先清缓存再用 waitForService 重连）；对系统服务改用 `ServiceManager.getService` 每次校验或 registerForNotifications（模板四）；不要长期缓存裸 IBinder 不挂失效逻辑。

场景三：oneway 洪泛——from BR_FAILED_REPLY 到 SPAM 点名

- **症状**：高频 oneway 调用方 logcat 出现 `ALOGE("Process seems to be sending too many oneway calls.")` 加 `oneway spamming` 调用栈（IPCThreadState.cpp:916-919）；或调用侧间歇性 FAILED_TRANSACTION 但单笔数据很小。
- **定位命令**：`logcat -b all | grep "oneway spamming"` 拿到发送方栈（谁在刷）；`cat /sys/kernel/debug/binder/state | grep -A5 async` 观察目标进程 async_free_space 水位；**dumpsys binder <对端>** 的 pending async 计数。
- **根因**：oneway 有独立预算（async 空间≈映射区一半，第三章 3.1 第 4 步），同一目标的 oneway 在驱动内排队保序；发送快于消费时队列线性增长，塞满后新事务被 `BR_FAILED_REPLY` 拒绝。13 起 `open_driver` 默认开启 BINDER_ENABLE_ONEWAY_SPAM_DETECTION（ProcessState.cpp:476），异常高频者被点名进日志——**只警告不拦截**（14.3）。
- **修复**：降频或合批（把 N 次 oneway 合成 1 次携带列表的事务）；payload 减肥（传 diff 而非全量）；真正需要背压的场景改同步调用让发送方自然限速；系统进程可临时关闭泛洪检测二分问题（进程内 setDefaultToggle）。

场景四：binder 线程耗尽——starved 日志与 16/32 预算

- **症状**：目标进程日志出现 `ALOGE("binder thread pool (%zu threads) starved for %lld ms", ...)`（IPCThreadState.cpp:571，饥饿超 100ms 记录）；或发送方出现 `ALOGW("Waiting for thread to be free. mExecutingThreadsCount=%lu mMaxThreads=%lu")`（:531，本进程 executing 打满等释放）；ANR trace 里所有 Binder:xxx_N 线程都在忙。
- **定位命令**：`debuggerd -b <pid>` 按 10.6 三类栈型分类（沉在 ioctl=空闲、停在 transact=出站外呼、挂在业务=入站服务）；`cat /sys/kernel/debug/binder/proc/<pid>` 数 threads 与每线程

transaction 栈；对"出站型"追第三方调用链，对"入站型"看 onTransact 里的慢业务；`dumpsys binder_calls_stats` 找最慢接口。

- **根因**：线程预算是供给制（默认 15+1，system_server 31+1，第四章），驱动按需 BR_SPAWN_LOOPER 扩池到上限为止；真正耗尽的几乎总是**嵌套同步调用**（A→B→C→…每跳占一线程，10.1 图的对称方向）或慢业务占坑——例如 binder 线程里做磁盘 IO/等待锁/同步外呼更慢的服务。
- **修复**：慢业务挪出 binder 线程（handler/post 到工作线程，事务即域——数据要复制走，第六章 6.1）；消除环状同步调用或改 oneway（注意 oneway 不携带返回值语义）；确需更大并发预算的专用服务可在 setThreadPoolMaxThreadCount 调整（别超 31 一倍，驱动 TF_ABORT 与调度成本随之涨）。

场景五：服务注册失败——SELinux 与 VINTF 两道关卡

- **症状**：`addService` 返回 SecurityException 或服务悄悄没上名录；`logcat -b events` 里 `avc: denied { add } for service=xxx pid=... scontext=u:r:vendor_xxx:s0 tcontext=u:object_r:default_android_service:s0 ...`；或 `service list` 里死活找不到名字。
- **定位命令**：`logcat -b events | grep avc` 拿完整 denied 向量（add/find）；**service list | grep <名>** 确认注册是否成功；native 侧直接看 ALOGE 拒绝日志（ServiceManager.cpp 五道关卡，第七章——哪一道在拒一目了然）；HAL 服务再核对 `/vendor/etc/vintf/manifest.xml` 是否声明了该实例。
- **根因**：名录是访问控制第一道门（第七章）：app uid 直接拒绝（ServiceManager.cpp:298）、SELinux canAdd 不放行（:302，add 权限在 service_contexts 标定的 tcontext 上）、HAL 名字与 VINTF 清单不一致（:317）。avc denied 的 scontext/tcontext 告诉你"谁在哪个标签上缺哪个权限"。
- **修复**：确属系统组件则补 sepolicy allow（service add 到正确的 service_contexts 标签，vendor 服务用 vendor_service 标签族）；HAL 服务核对 manifest 与 VINTF 一致性再注册；**永不**用 permissive 糊弄量产——五道关卡的存在理由见 1.4 能力模型。

场景六：调用卡死十秒——同步事务的去向追踪

- **症状**：App 或 native 服务在 transact 上无限期挂起（Binder 默认无超时！），表现为 ANR（`Input dispatching timed out`）或功能假死；`debuggerd -b` 显示线程停在 `IPCThreadState::waitForResponse` → `ioctl`。
- **定位命令**：先抓发送方栈确认挂在哪个事务（**debuggerd -b <发起方>** 栈顶 android_os_BinderProxy_transact 下面的业务帧即是接口名）；再抓对端 **debuggerd -b <对端>**，在 `cat /sys/kernel/debug/binder/transactions` 找 `node ... -> proc ...` 的 pending 记录，能看到"谁押着谁"（12.1：事务栈里有别人=嵌套调用现场）；perfetto binder_track 把"唤醒-执行-回复"三段时长拆开（9.3 五步法）。
- **根因**：同步事务没有协议级超时（UAPI 无 per-transaction timeout 字段），卡死=对端 binder 线程永不回 BC_REPLY。三大病灶：对端慢业务占满线程池（场景四的伴生症状）；**循环等待**

——A 的 binder 线程持锁同步调 B，B 的业务又要等 A 释放的锁（跨进程死锁，thread 排队不回滚）；对端服务在等一个永远不会来的依赖（初始化顺序错误）。

- **修复**：业务层自建超时（调用侧包 future.get(timeout)，注意超时后事务仍在路上——幂等设计）；打破循环等待（对可疑回调用 oneway + 回执分离，或先释放锁再外呼——clearCallingIdentity 不救这个，锁序才是关键）；对冻结对端预期 BR_FROZEN_REPLY 快速失败（10.4）而非挂起。协议层没有 per-transaction 超时字段，超时永远是业务层的责任。